



AGRUPAMENTO DE ESCOLAS
EDUARDO GAGEIRO

Escola Secundária de Sacavém

**CURSO PROFISSIONAL DE GESTÃO E
PROGRAMAÇÃO DE SISTEMAS INFORMÁTICOS**

PROVA DE APTIDÃO PROFISSIONAL

Relatório de Projeto

Ano Letivo 2025/2026

Relatório de Projeto

ARMIAc – Braço Inteligente



Quévin Leonardo Aguiar Tavares

2025/2026

Professoras Orientadoras: Cláudia Barata & Paula Alcobia

Índice

Agradecimentos.....	6
1 – Introdução.....	7
1.1 – Objetivo e aplicação real no ensino.....	8
2 – Tecnologias Envolvidas.....	9
2.1 – Software utilizado.....	9
2.2 – Dispositivos utilizados.....	10
2.3 – Linguagens de Programação.....	11
2.4 – Tecnologias e comunicação.....	12
2.5 – Componentes eletrônicos utilizados.....	12
2.6 – Impressão 3D e materiais de montagem.....	17
3 – Implementação do Projeto.....	20
4 – Desenvolvimento do Projeto.....	21
4.1 – Cronograma e fases de desenvolvimento.....	21
4.2 – Elaboração do protótipo.....	28
4.2.1 – Criação dos modelos 3D.....	28
4.2.2 – Preparação dos modelos 3D.....	28
4.2.3 – Impressão 3D das peças.....	31
4.2.4 – Montagem dos dedos e articulações.....	32
4.2.5 – Montagem da mão e do pulso.....	33
4.2.6 – Instalação dos servomotores do antebraço.....	34
4.2.7 – Cabos e extensões.....	37
4.2.8 – Encaixe da mão e posicionamento dos fios.....	38
4.2.9 – Criação da base do ARMIAC.....	39
4.2.10 – Testes elétricos e medições com multímetro.....	39
4.2.11 – Ligações entre ESP32 e PCA9685.....	40
4.2.12 – Alimentação e primeiros testes.....	41
4.3 – Programação do ESP32 (Arduino IDE).....	42
4.4 – Programação da aplicação Flutter.....	42
4.4.1 – Criação do projeto Flutter.....	44
4.4.2 – Gestão de dependências e recursos (pubspec.yaml).....	45
4.4.3 – Organização do projeto Flutter (estrutura de pastas).....	46
4.4.4 – Geração do APK da aplicação.....	47
4.5 – Identidade Visual do Projeto (Logótipo ARMIAC).....	47
4.6 – Dificuldades encontradas durante o desenvolvimento.....	47
5 – Conclusão.....	49
5.1 – Melhorias futuras.....	50
6 – Webgrafia.....	51
7 – Anexos.....	53
7.1 - Código.....	53
7.2 – APK da aplicação ARMIAC.....	136
7.3 – Links dos vídeos do Youtube.....	136

Índice de Figuras

Figura 1: Braço robótico ARMIAC.....	8
Figura 2: Braço robótico ARMIAC a segurar um objeto.....	8
Figura 3: Arduino IDE utilizado para programação do ESP32.....	9
Figura 4: Visual Studio Code utilizado para programação da aplicação Flutter.....	10
Figura 5: Criação do logótipo ARMIAC no Canva.....	10
Figura 6: Samsung S21 utilizado para a APP.....	10
Figura 7: PC utilizado para programação.....	10
Figura 8: Portátil usado para programação.....	11
Figura 9: Impressora 3D Elegoo Neptune 4.....	11
Figura 10: Micro-controlador ESP32.....	12
Figura 11: Módulo PCA9685 (Driver PWM 16 canais).....	13
Figura 12: Servo MZ996R.....	13
Figura 13: Servo JX (baixo perfil).....	14
Figura 14: Fonte de alimentação DC 6V 10A.....	14
Figura 15: Medição de tensão com multímetro antes de ligar o sistema.....	15
Figura 16: Cabos de extensão utilizados para ligação do servo do polegar.....	16
Figura 17: Cabos de extensão utilizados para ligação do servo do polegar em uso.....	16
Figura 18: Impressora Elegoo Neptune 3D.....	17
Figura 19: Materiais de montagem utilizados no projeto.....	18
Figura 20: Mapa Representativo.....	20
Figura 21: Primeiro contacto com este tipo de projeto.....	21
Figura 22: Impressão de peças 3D.....	22
Figura 23: Início da montagem da mão.....	23
Figura 24: Primeiro teste com servo.....	24
Figura 25: Parte interna do braço.....	25
Figura 26: Criação da Mão no TinkerCAD.....	25
Figura 27: Braço montado.....	26
Figura 28: App no telemóvel – versão antiga.....	27
Figura 29: Programação da App em Dart.....	27
Figura 30: Criação do cotovelo no TinkerCAD.....	28
Figura 31: Base da mão.....	28
Figura 32: Dedos.....	29

Figura 33: Dedo opositor.....	29
Figura 34: Pulso.....	29
Figura 35: Antebraço.....	29
Figura 36: Cotovelo.....	30
Figura 37: Suporte dos servos.....	30
Figura 38: Attach dos servos.....	30
Figura 39: Cover do antebraço.....	30
Figura 40: Erro na impressão do antebraço.....	31
Figura 41: Montagem dos dedos.....	32
Figura 42: Dedos montados na base da mão.....	32
Figura 43: Inserção do servo do polegar.....	33
Figura 44: Encaixe do dedo opositor na mão.....	33
Figura 45: Fio de nylon para puxar os dedos.....	33
Figura 46: Montagem do pulso.....	34
Figura 47: Montagem da mão finalizada.....	34
Figura 48: Servos montados no suporte.....	34
Figura 49: Horn do servo com os anexos criados em 3D.....	34
Figura 50: Encaixe do horn com attach nos servos.....	35
Figura 51: Horn do servo do antebraço.....	35
Figura 52: Montagem do suporte de servo no antebraço.....	35
Figura 53: Distribuição dos servomotores pelos canais PWM do PCA9685.....	36
Figura 54: Cabos de extensão utilizados para ligação do servo do polegar.....	37
Figura 55: Fios de nylon encaixados nos attachs dos servos.....	38
Figura 56: Base do Armiac.....	39
Figura 57: Medição de tensão com multímetro antes de ligar o sistema.....	39
Figura 58: Esquema de ligação entre o ESP32 e o módulo PCA9685.....	40
Figura 59: Ligação da fonte de alimentação ao PCA9685.....	41
Figura 60: Página inicial da app.....	42
Figura 61: Slides de mini-tutorial da app - 1.....	42
Figura 62: Slides de mini-tutorial da app - 2.....	43
Figura 63: Painel de controlos na app.....	43
Figura 64: Lista de comandos por voz possíveis.....	43
Figura 65: Página de configurações.....	43
Figura 66: Página de termos de serviço.....	43

Figura 67: Criação do projeto flutter.....44

Figura 68: Estrutura das pastas.....46

Figura 69: Logo ARMIAC.....47

Figura 70: Projeto Finalizado.....50

Agradecimentos

Em primeiro lugar, gostaria de agradecer às professoras orientadoras **Cláudia Barata** e **Paula Alcobia**, pelo acompanhamento, apoio e sugestões ao longo do desenvolvimento do projeto.

Agradeço também a todos os professores que contribuíram para a minha formação ao longo do Curso Profissional de Gestão e Programação de Sistemas Informáticos, fornecendo conhecimentos importantes para a realização deste trabalho, nomeadamente ao professor **Luís Roque** por fornecer acesso à sala de impressão 3D disponível na escola.

Não posso deixar de agradecer à minha família, especialmente à minha mãe pelo apoio e incentivo durante todo o processo de desenvolvimento do projeto.

Por fim, agradeço a todos os colegas e pessoas que de alguma forma contribuíram para o desenvolvimento do ARMIAC, através de opiniões, ideias e feedback.

1 – Introdução

A PAP, Prova de Aptidão Profissional, é um projeto final de curso que tem como objetivo demonstrar as competências adquiridas ao longo dos três anos de formação no Curso Profissional de Gestão e Programação de Sistemas Informáticos.

O projeto escolhido para a apresentação da PAP intitula-se “**ARMAC – Braço Inteligente**”. A escolha deste tema surgiu devido ao meu interesse pela robótica, programação, eletrónica e simulação de movimentos humanos.

O ARMAC consiste numa mão robótica impressa em 3D, controlada por servomotores e por uma aplicação móvel desenvolvida em Flutter. A aplicação permite enviar comandos para abrir e fechar a mão, controlar movimentos individuais e executar diferentes gestos através de uma ligação Wi-Fi, tanto por voz como por botões.

O sistema utiliza um ESP32 como controlador principal, responsável por receber os comandos enviados pela aplicação, e um módulo PCA9685 para controlar os servomotores. Desta forma, os comandos enviados pelo utilizador na aplicação são transformados em movimentos físicos da mão robótica.

A aplicação móvel foi desenvolvida para Android e inclui uma interface simples e organizada, com botões de controlo, comandos rápidos, painel de movimentos, comandos por voz, tutorial de utilização, suporte em português e inglês, lista de comandos disponíveis e página de termos de serviço. A última versão do projeto é a 1.1.7.

Este projeto foi desenvolvido em várias fases, desde a pesquisa inicial até aos testes finais do protótipo. Inicialmente foi realizado o planeamento do projeto e o estudo de soluções semelhantes. De seguida, foram criadas e impressas as peças em 3D, montada a estrutura física, instalados os servomotores e realizadas as ligações eletrónicas. Por fim, foi feita a programação do ESP32, o desenvolvimento da aplicação móvel e os testes de funcionamento.

Ao longo deste relatório serão apresentadas as tecnologias utilizadas, os componentes aplicados, o processo de desenvolvimento do protótipo, a implementação do sistema, as dificuldades encontradas, os resultados obtidos e possíveis melhorias futuras.

Figura 1: Braço robótico ARMIAC

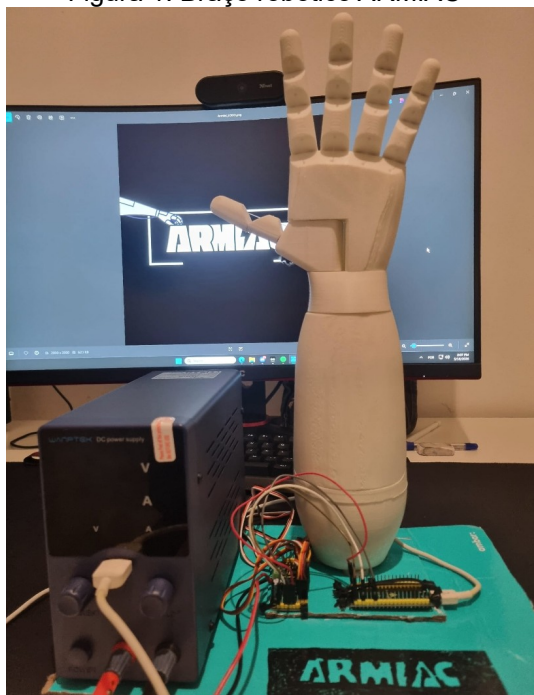
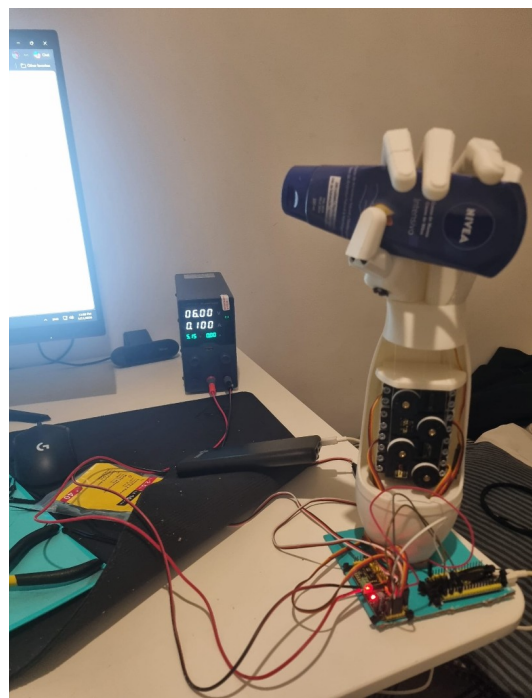


Figura 2: Braço robótico ARMIAC a segurar um objeto



1.1 – Objetivo e aplicação real no ensino

O objetivo principal do ARMIAC é desenvolver um protótipo funcional de uma mão robótica controlada por uma aplicação móvel, demonstrando a ligação entre programação, eletrónica, robótica, impressão 3D e desenvolvimento de aplicações.

Numa aplicação real no ensino, o ARMIAC poderia ser utilizado como uma ferramenta **educativa** em escolas, cursos profissionais, clubes de robótica ou formações tecnológicas. Através deste projeto, os alunos poderiam aprender de forma prática como funciona a comunicação entre software e hardware, percebendo como uma aplicação móvel consegue enviar comandos para um microcontrolador e controlar movimentos físicos através de servomotores.

O projeto também permite explorar conteúdos de várias áreas, como programação em Dart e C++, desenvolvimento de aplicações Flutter, redes Wi-Fi, comunicação HTTP, controlo PWM, modelação 3D, impressão 3D e montagem eletrónica. Assim, o ARMIAC pode servir como exemplo prático para aulas de programação, eletrónica e robótica, tornando a aprendizagem mais visual, interativa e próxima de situações reais.

Além disso, por ser um protótipo físico, o ARMIAC facilita a compreensão dos erros e desafios encontrados durante o desenvolvimento de um sistema tecnológico, como problemas de alimentação, ligações elétricas, calibração de servos, organização de código e testes finais. Desta forma, o projeto não serve apenas para demonstrar um produto final, mas também para mostrar todo o processo de criação, teste e melhoria de uma solução tecnológica.

2 – Tecnologias Envolvidas

Para a elaboração deste projeto foi necessário utilizar diversas tecnologias e ferramentas, tanto ao nível de software como de hardware.

2.1 – Software utilizado

Software para escrita do relatório:

- LibreOffice

Software para programação (IDE):

- Arduino IDE
- Visual Studio Code / Android Studio

Software para criação dos modelos em 3D:

- TinkerCAD

Software para impressão 3D:

- Elegoo Cura/Ultimaker Cura

Software para criação gráfica:

- Canva (logótipo do ARMIAC e mapa representativo)

Figura 3: Arduino IDE utilizado para programação do ESP32

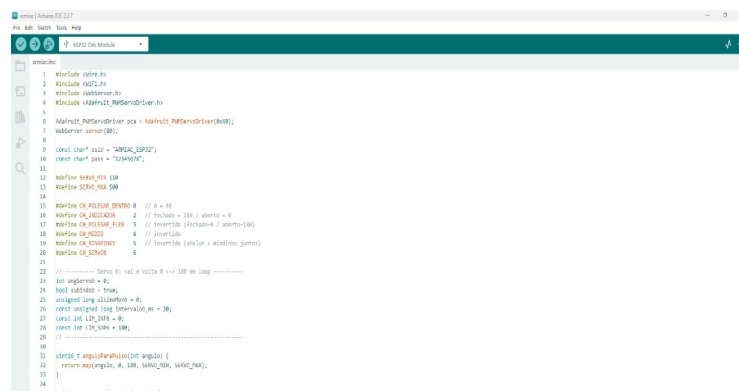


Figura 4: Visual Studio Code utilizado para programação da aplicação Flutter

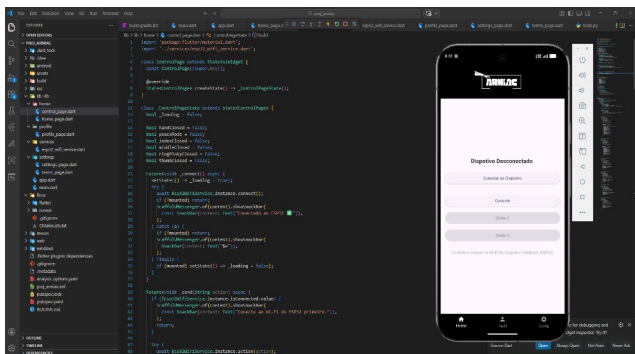
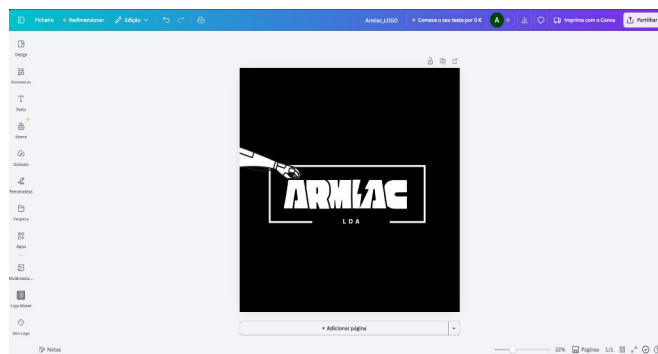


Figura 5: Criação do logótipo ARMIAC no Canva



2.2 – Dispositivos utilizados

- **Computador desktop e computador portátil** – utilizado para programar o ESP32, desenvolver a aplicação Flutter, escrever o relatório e preparar os ficheiros 3D;
- **Telemóvel Samsung S21** – utilizado para testar a aplicação ARMIAC e enviar comandos ao braço robótico;
- **Impressora 3D Elegoo Neptune 4** – utilizada para imprimir as peças da mão, antebraço e base;

Figura 6: Samsung S21 utilizado para a APP



Figura 7: PC utilizado para programação



Figura 8: Portátil usado para programação



Figura 9: Impressora 3D Elegoo Neptune 4



2.3 – Linguagens de Programação

- **C++** (Arduino / ESP32) - utilizada na programação do ESP32 através do Arduino IDE. Esta linguagem foi usada para criar a rede Wi-Fi do ARMIAC, receber comandos HTTP, controlar o PCA9685 e executar os movimentos dos servomotores.
- **Dart** (Flutter) - utilizada no desenvolvimento da aplicação móvel com Flutter. Esta linguagem permitiu criar a interface gráfica, o sistema de comandos, o controlo por voz, a tradução da aplicação para português e inglês e a comunicação com o ESP32.

2.4 – Tecnologias e comunicação

O ARMIAC utiliza diferentes tecnologias de comunicação para ligar a aplicação móvel ao sistema físico.

- **Wi-Fi** - O **ESP32** cria uma rede Wi-Fi própria em modo Access Point, chamada “ARMIAC”. O telemóvel liga-se diretamente a essa rede, sem necessidade de router ou internet.
- **Protocolo HTTP** - A aplicação móvel envia comandos ao ESP32 através do protocolo HTTP. Estes comandos são enviados por URLs, como por exemplo (“/action?a=open_hand”), que indica ao ESP32 que deve abrir a mão.
- **I2C/PWM** - Entre o ESP32 e o PCA9685 é utilizada comunicação **I2C**, através dos pinos SDA e SCL. O PCA9685 recebe as instruções do ESP32 e gera os sinais PWM necessários para controlar os servomotores.

2.5 – Componentes eletrónicos utilizados

ESP32

Microcontrolador utilizado como controlador principal do sistema, responsável por criar a rede Wi-Fi do ARMIAC, receber os comandos enviados pela aplicação móvel e comunicar com o controlador PCA9685.

Funciona como o “cérebro” do sistema, mas não controla diretamente todos os servomotores. Em vez disso, envia instruções ao PCA9685, que é responsável por gerar os sinais PWM para os servos.

Figura 10: Micro-controlador ESP32



PCA9685

O **PCA9685** é um controlador PWM de 16 canais utilizado para controlar vários servomotores de forma mais estável. Como o projeto utiliza vários servos para movimentar os dedos e o antebraço, o PCA9685 facilita a organização das ligações.

Figura 11: Módulo PCA9685 (Driver PWM 16 canais)



Servomotores

Os servomotores são responsáveis pelo movimento físico dos dedos e do antebraço. Cada servo recebe um sinal PWM e posiciona-se num determinado ângulo, permitindo abrir ou fechar os dedos.

Servo MZ996R

Servo utilizado para flexionar os dedos e rotacionar o antebraço.

Figura 12: Servo MZ996R



Servo JX (baixo perfil)

Servo utilizado especificamente para o movimento do polegar para dentro.

Figura 13: Servo JX (baixo perfil)



Fonte DC 6V 10A

Fonte utilizada para alimentar todos os servomotores e garantir estabilidade de corrente.

Figura 14: Fonte de alimentação DC 6V 10A



Multímetro

Durante o desenvolvimento do projeto foi utilizado um multímetro digital para medir a voltagem e garantir segurança antes de ligar os componentes à fonte de alimentação.

Figura 15: Medição de tensão com multímetro antes de ligar o sistema



Para a montagem e ligação dos servomotores foi necessário utilizar cabos e extensões, garantindo uma ligação correta entre os servos e o PCA9685.

Foram utilizados cabos de extensão de servo principalmente para o servo low profile do polegar, pois o posicionamento deste servo exigiu maior comprimento de cabo.

Figura 16: Cabos de extensão utilizados para ligação do servo do polegar

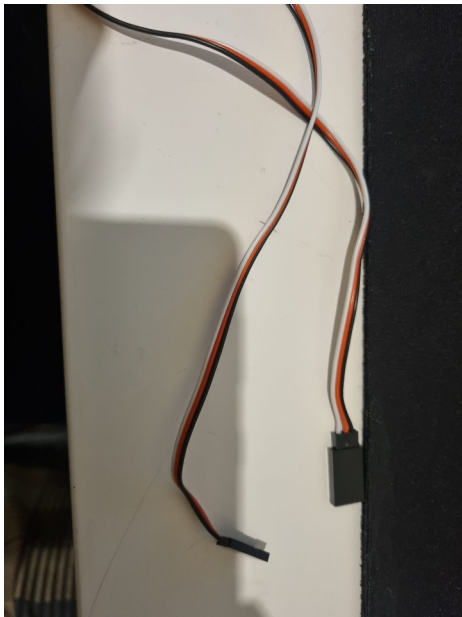
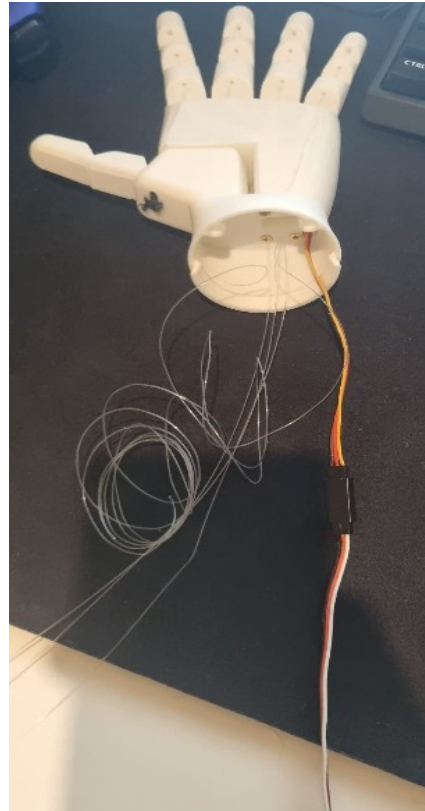


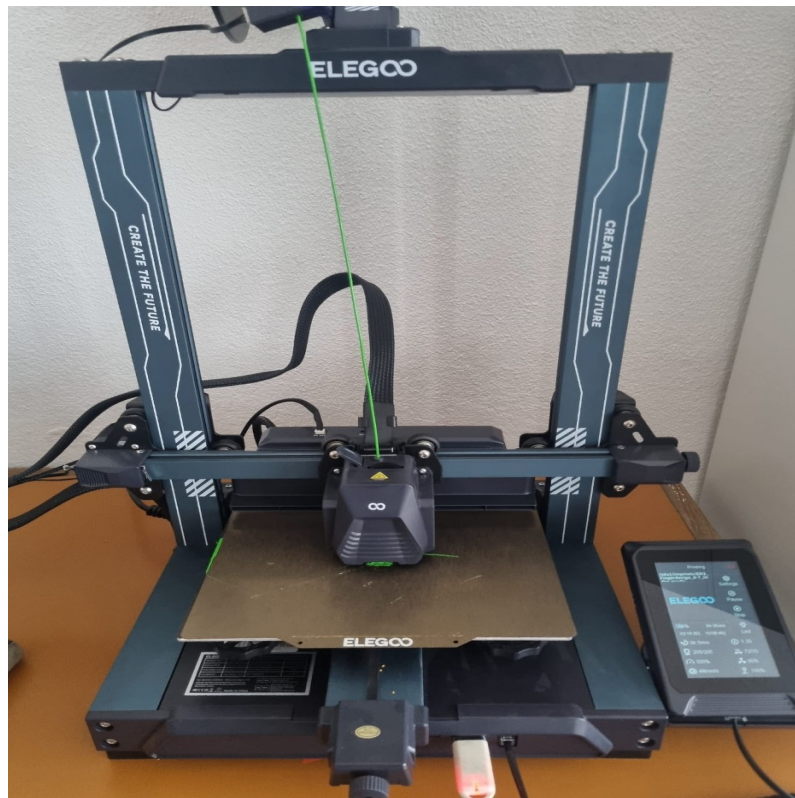
Figura 17: Cabos de extensão utilizados para ligação do servo do polegar em uso



2.6 – Impressão 3D e materiais de montagem

O protótipo foi construído com peças impressas em 3D numa impressora Elegoo Neptune 4 a utilizar o material PLA.

Figura 18: Impressora Elegoo Neptune 3D



Materiais utilizados:

- Fio elástico preto 3 mm
- Fio de nylon 0,4 mm
- Parafusos
- Supercola

Figura 19: Materiais de montagem utilizados no projeto



Componente / Material	Quantidade aproximada	Função no projeto
ESP32	1	Controlador principal do sistema. Cria a rede Wi-Fi, recebe comandos da aplicação e comunica com o PCA9685.
PCA9685	1	Controlador PWM utilizado para controlar vários servomotores de forma organizada e estável.
Servomotores MZ996R	6	Responsáveis por movimentar os dedos e o antebraço.
Servo JX low profile	1	Utilizado no movimento do polegar para dentro da mão.
Fonte DC 6V 10A	1	Alimenta os servomotores com energia suficiente e estável.
Cabos de extensão de servo	1	Permitem ligar servos mais afastados ao PCA9685.
Fio de nylon 0,4 mm	Conforme necessário	Usado para puxar os dedos, simulando tendões.
Fio elástico preto 3 mm	Conforme necessário	Usado nas articulações dos dedos para ajudar no movimento.
Parafusos	32	Fixação das peças impressas e dos componentes.
Supercola	1	Fixação de algumas peças da estrutura.
PLA branco	Conforme necessário	Material usado na impressão 3D das peças.
Impressora 3D Elegoo Neptune 4	1	Impressão das peças da mão, antebraço e base.
Telemóvel Samsung S21	1	Teste e utilização da aplicação ARMIAC.
Computador / portátil	1 ou mais	Programação do ESP32, desenvolvimento da app e preparação dos ficheiros 3D.
Multímetro digital	1	Medição da tensão da fonte e verificação das ligações elétricas.

3 – Implementação do Projeto

Enquadramento Teórico

O funcionamento do ARMIAC pode ser dividido em quatro partes principais: aplicação móvel, ESP32, PCA9685 e servomotores.

Braço robótico

Um braço robótico é um sistema mecânico e eletrónico capaz de simular movimentos humanos. Estes sistemas são frequentemente utilizados em áreas industriais, médicas e tecnológicas.

No projeto ARMIAC foi desenvolvido um protótipo de braço robótico com mão articulada, permitindo diversos movimentos com a mão e antebraço.

Servomotores

Os servomotores permitem controlar um ângulo com precisão, geralmente entre 0° e 180°. O ARMIAC utiliza servos para controlar os movimentos de cada dedo.

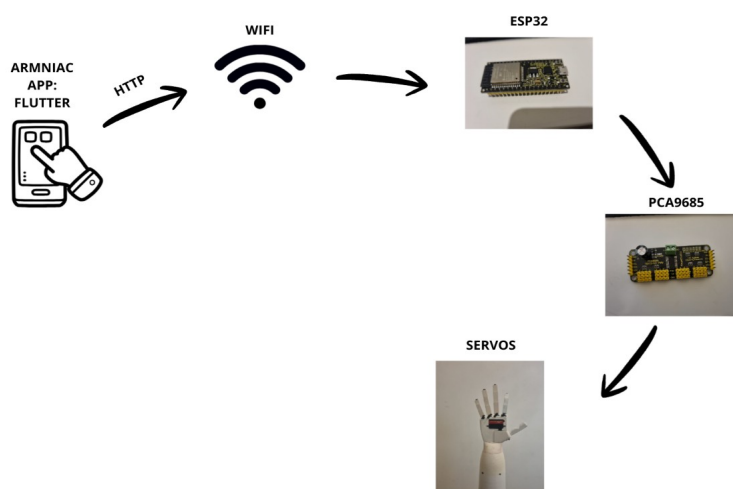
PWM

PWM (Pulse Width Modulation) é uma técnica utilizada para controlar a posição dos servos através de sinais elétricos com larguras de pulso diferentes.

Comunicação por Wi-Fi e HTTP

O controlo remoto é feito através de uma rede Wi-Fi criada pelo ESP32. A aplicação Flutter envia comandos HTTP que são interpretados pelo microcontrolador.

Figura 20: Mapa Representativo



4 – Desenvolvimento do Projeto

4.1 – Cronograma e fases de desenvolvimento

O desenvolvimento do projeto **ARMIAc – Braço Inteligente** foi realizado ao longo do ano letivo 2025/2026, seguindo uma sequência de etapas organizadas, desde a fase inicial de pesquisa até ao desenvolvimento da aplicação móvel e testes finais do protótipo.

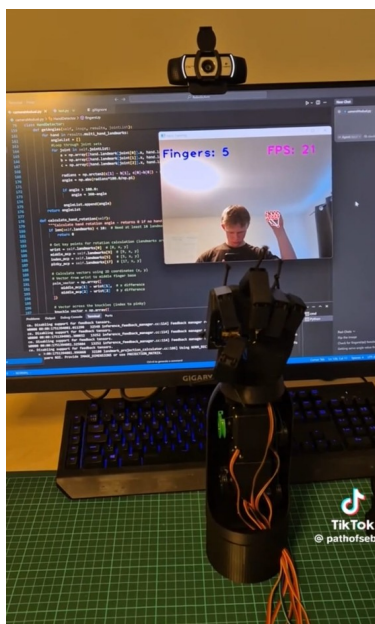
O projeto foi planeado de forma progressiva, permitindo primeiro estruturar a ideia e estudar possíveis soluções técnicas, criando as peças e em seguida avançando para a impressão 3D, montagem física e eletrónica, e por fim para a implementação do software.

As fases principais do projeto foram as seguintes:

Fase 1 – Pesquisa e planeamento (05 de outubro)

No dia **05 de outubro**, iniciou-se a fase de pesquisa e planeamento, onde foram analisadas ideias e estudados projetos semelhantes, com o objetivo de compreender os componentes necessários e como seria possível desenvolver uma mão robótica funcional.

Figura 21: Primeiro contacto com este tipo de projeto



Demonstração de dois projetos base em “Exemplo dos projetos base” nos anexos, na secção 7.3

Fase 2 – Impressão das peças 3D – 1.ª versão (20 de outubro)

No dia **20 de outubro**, iniciou-se a impressão 3D das peças do projeto. Nesta fase foram produzidas as primeiras estruturas necessárias para a montagem da mão e antebraço, utilizando 2 impressoras Elegoo Neptune 4.

Nesta fase, foi utilizada uma primeira versão de peças 3D encontradas na internet, que serviram como base inicial para compreender melhor a estrutura de uma mão robótica e o funcionamento das articulações. Esta versão permitiu ter um primeiro contacto com o formato das peças, com os encaixes e com a montagem geral do protótipo.

No entanto, após imprimir e analisar o resultado, verificou-se que a versão inicial não correspondia totalmente ao que pretendia para o ARMIAC. Por esse motivo, foi decidido criar uma versão própria das peças, usando a primeira versão apenas como referência.

Figura 22: Impressão de peças 3D



Demonstração em “Impressão 3D” nos anexos, na secção 7.3

Fase 3 – Montagem inicial da mão (29 de outubro)

No dia **29 de outubro**, iniciou-se a montagem da mão robótica, incluindo a montagem das articulações dos dedos e o ajuste inicial das peças impressas.

Figura 23: Início da montagem da mão



Fase 4 – Primeiro teste com servomotor (05 de novembro)

No dia **05 de novembro**, foi realizado o primeiro teste com um mini servomotor, com o objetivo de validar o funcionamento mecânico do sistema e testar a capacidade de movimentação de um dedo.

Este teste foi essencial para confirmar que o mecanismo de fios e articulações funcionava corretamente.

Figura 24: Primeiro teste com servo



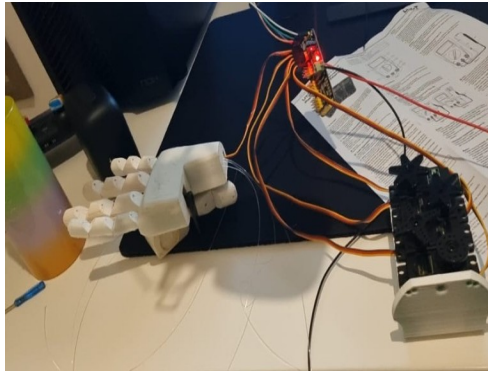
Demonstração em “Primeiro teste com servomotor” nos anexos, na secção 7.3

Fase 5 – Testes e montagem do antebraço (14 a 16 de novembro)

Entre os dias **14 e 16 de novembro**, foram realizados vários testes adicionais e iniciou-se a montagem do restante antebraço, incluindo a instalação de mais servos e a organização do espaço interno.

Durante esta fase foram feitos ajustes no sistema de fios e nas ligações elétricas, garantindo maior estabilidade e funcionalidade do protótipo.

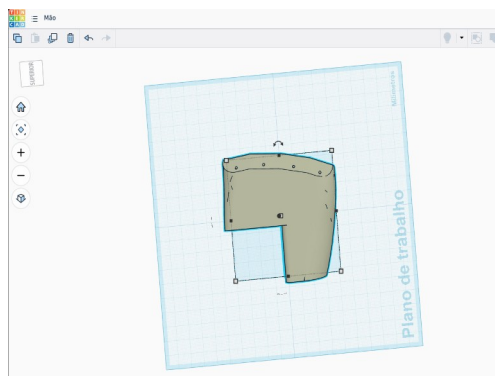
Figura 25: Parte interna do braço



Fase 6 – Criação e impressão de uma versão própria das peças 3D (29 de novembro a finais de dezembro)

Entre o dia **29 de novembro** e o **período do Natal**, foi iniciada a criação de uma versão própria das peças 3D do ARMIAC, utilizando como referência a primeira versão encontrada na internet. Após os primeiros testes, verificou-se que era necessário adaptar as peças ao projeto, melhorando os encaixes, os dedos e a passagem dos fios. Também foi decidido imprimir as peças em PLA branco, uma vez que o resultado visual ficou mais limpo e adequado ao protótipo final.

Figura 26: Criação da Mão no TinkerCAD



Fase 7 – Montagem final do antebraço (20 de janeiro)

No dia **20 de janeiro**, o antebraço encontrava-se já totalmente montado, incluindo a instalação dos servomotores, fixação das peças e preparação para testes finais e calibração.

Figura 27: Braço montado



Fase 8 – Desenvolvimento da aplicação móvel (21 de janeiro)

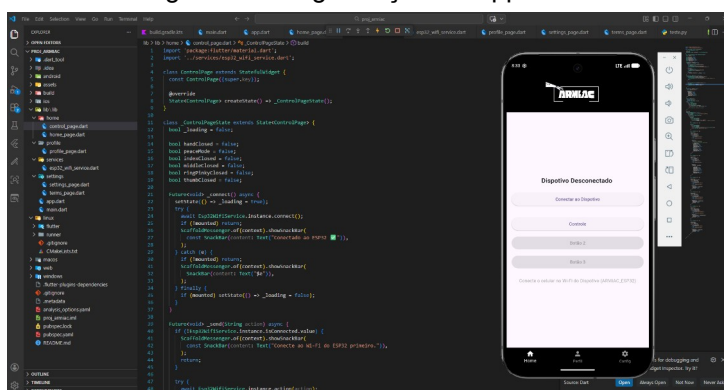
No dia **21 de janeiro**, iniciou-se o desenvolvimento da aplicação móvel em Flutter, com o objetivo de criar uma interface simples e funcional para o controlo remoto do braço robótico através de Wi-Fi.

Nesta fase foram criadas páginas de controlo, botões de ações e um sistema de comunicação HTTP entre o telemóvel e o ESP32.

Figura 28: App no telemóvel – versão antiga



Figura 29: Programação da App em Dart



No dia **17 de maio** foi realizada a demonstração do protótipo ARMIAC juntamente com a aplicação móvel desenvolvida em Flutter. Nesta fase foram efetuados testes finais ao sistema completo, verificando a comunicação Wi-Fi entre o telemóvel e o ESP32 e a resposta dos servomotores aos comandos enviados pela aplicação.

Após a validação do funcionamento do protótipo, foi gravado um vídeo de demonstração onde é possível observar o controlo do braço robótico através da aplicação e a execução dos movimentos e gestos configurados.

Demonstração em “Demonstração do projeto inacabado” nos anexos, na secção 7.3

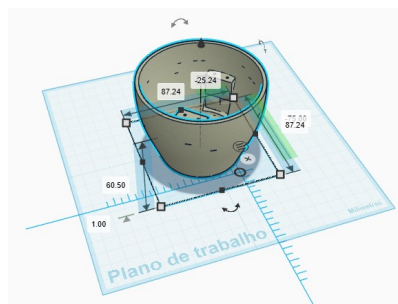
4.2 – Elaboração do protótipo

4.2.1 – Criação dos modelos 3D

Antes da impressão, foi feita a modelação 3D de algumas peças do ARMIAC. Nesta fase, foi utilizado o TinkerCAD , uma ferramenta online de modelação 3D.

A criação das peças teve como base a observação de projetos semelhantes encontrados online, que serviram como referência para perceber melhor a estrutura de uma mão robótica. No entanto, as peças foram adaptadas às necessidades do ARMIAC, tendo em conta o tamanho dos servomotores, a passagem dos fios, os encaixes e a montagem final.

Figura 30: Criação do cotovelo no TinkerCAD



4.2.2 – Preparação dos modelos 3D

Nesta fase foram ajustados e preparados modelos 3D para impressão das peças necessárias para o projeto, incluindo a estrutura da mão e do antebraço em formato STL. Os ficheiros foram posteriormente exportados em formato g-code.

Figura 31: Base da mão

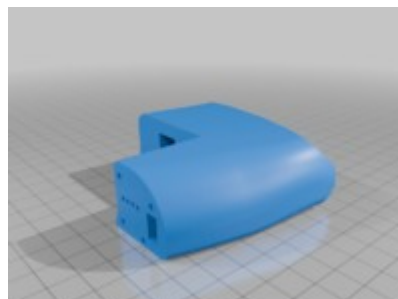


Figura 32: Dedos

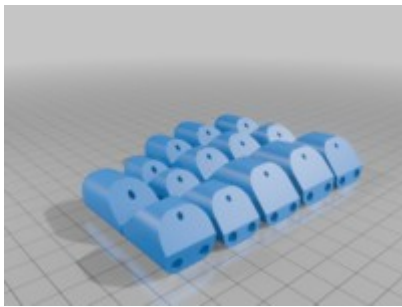


Figura 33: Dedo opositor

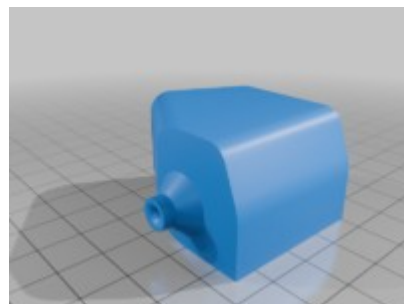


Figura 34: Pulso

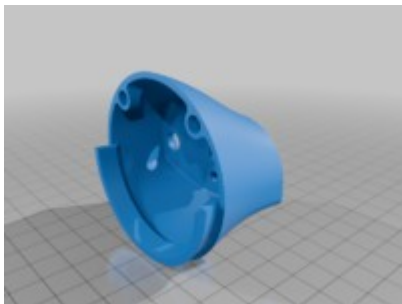


Figura 35: Antebraço

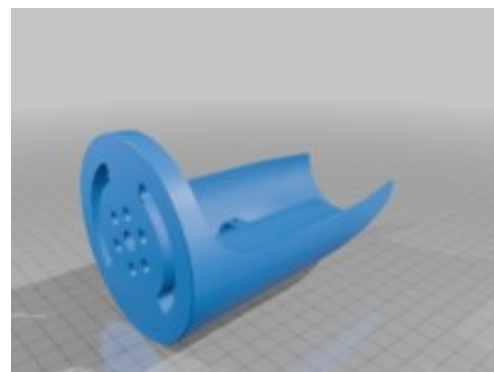


Figura 36: Cotovelo

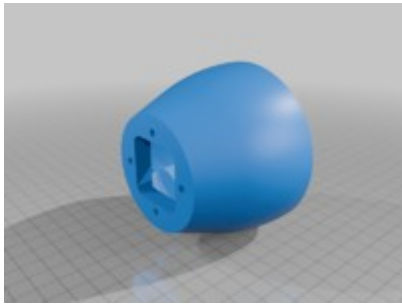


Figura 37: Suporte dos servos

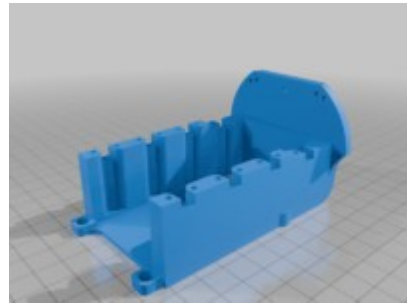


Figura 38: Attach dos servos

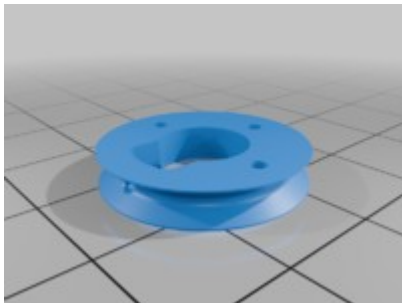
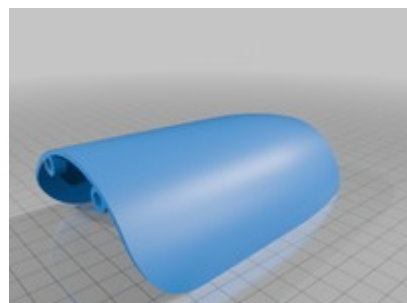


Figura 39: Cover do antebraço



4.2.3 – Impressão 3D das peças

Após a criação e preparação dos modelos 3D, foi realizada a impressão das peças utilizando a impressora Elegoo Neptune 4.

Figura 40: Erro na impressão do antebraço



4.2.4 – Montagem dos dedos e articulações

Os dedos foram montados com um fio elástico de 3mm que permite flexão, simulando o movimento natural de uma mão humana.

Figura 41: Montagem dos dedos

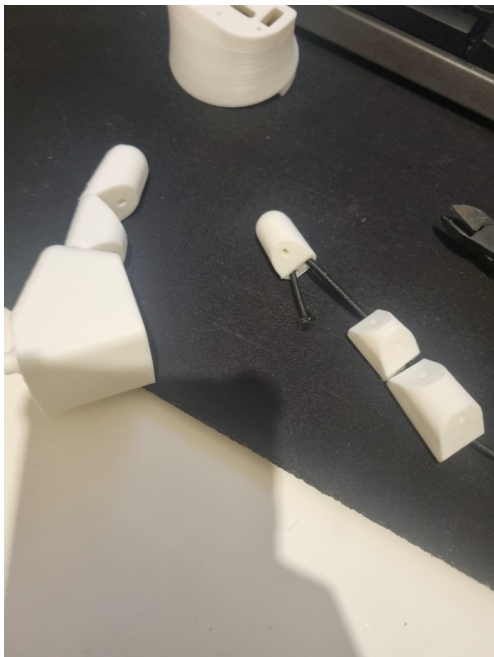


Figura 42: Dedos montados na base da mão



Demonstração em “Demonstração do funcionamento dos dedos” nos anexos, na secção 7.3

4.2.5 – Montagem da mão e do pulso

Foram passados os fios de nylon por dentro dos dedos e foi encaixado o servo do polegar.

Figura 43: Inserção do servo do polegar



Figura 44: Encaixe do dedo opositor na mão



Figura 45: Fio de nylon para puxar os dedos

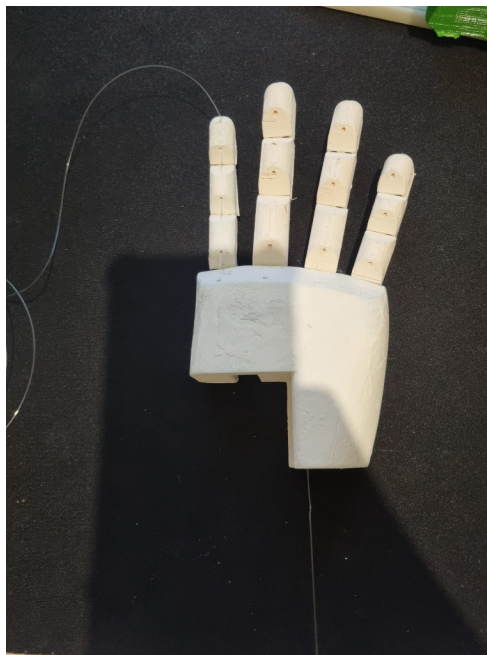
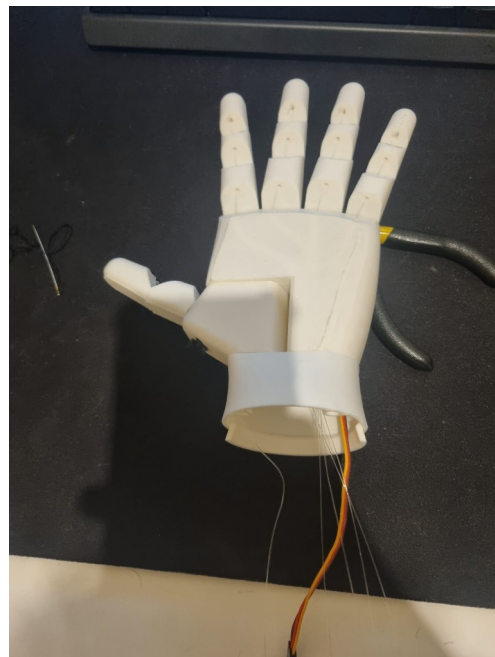


Figura 46: Montagem do pulso



Figura 47: Montagem da mão finalizada



4.2.6 – Instalação dos servomotores do antebraço

Foram instalados vários servomotores MZ996R para controlo dos dedos e rotação do antebraço, bem como um servo low profile para o polegar.

Figura 48: Servos montados no suporte



Figura 49: Horn do servo com os anexos criados em 3D



Figura 50: Encaixe do horn com attach nos servos



Figura 51: Horn do servo do antebraço

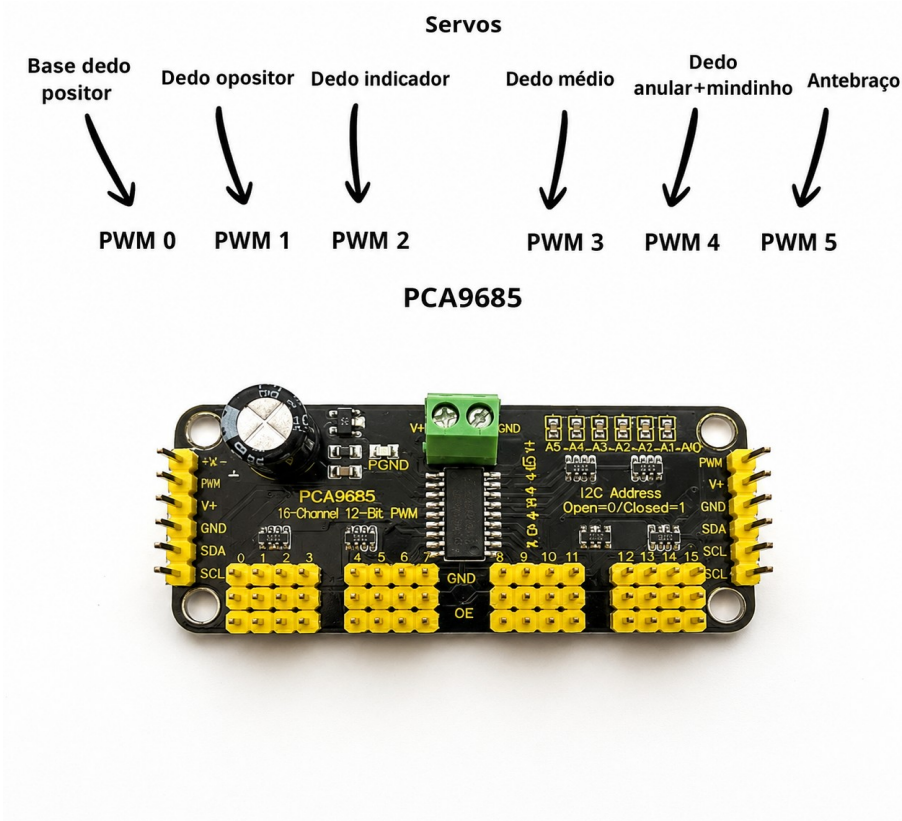


Figura 52: Montagem do suporte de servo no antebraço



Os servomotores foram ligados aos canais PWM do PCA9685 de forma organizada, permitindo controlar individualmente cada movimento da mão robótica. O canal PWM 0 foi associado à base do dedo opositor, o PWM 1 ao dedo opositor, o PWM 2 ao dedo indicador, o PWM 3 ao dedo médio, o PWM 4 ao dedo anelar e mindinho, e o PWM 5 ao movimento do antebraço. Esta organização facilitou a programação dos movimentos no ESP32 e tornou mais simples identificar cada servo durante os testes.

Figura 53: Distribuição dos servomotores pelos canais PWM do PCA9685



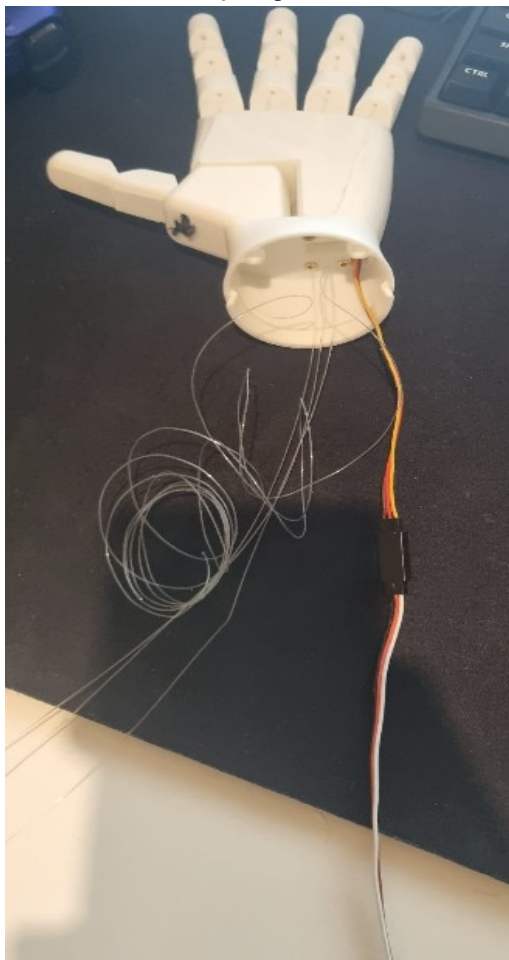
Canal no PCA9685	Movimento controlado
PWM 0	Base do dedo opositor
PWM 1	Dedo opositor
PWM 2	Dedo indicador
PWM 3	Dedo médio
PWM 4	Dedo anelar + mindinho
PWM 5	Antebraço

4.2.7 – Cabos e extensões

Devido à distância entre alguns servos e o controlador PCA9685, foram utilizadas extensões de cabos de servo, especialmente para o servo low profile do polegar.

Esta etapa foi importante para garantir estabilidade nas ligações e organização interna do braço.

Figura 54: Cabos de extensão utilizados para ligação do servo do polegar



4.2.8 – Encaixe da mão e posicionamento dos fios

Nesta etapa, foi encaixada a mão ao antebraço e foram passados os fios através dos espaços do suporte do servo, fixando-os e enrolando-os aos servos.

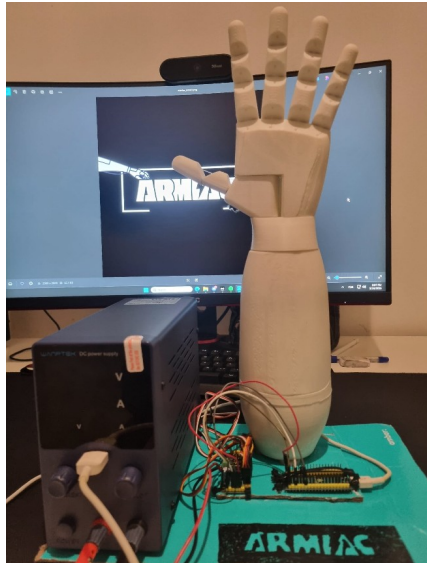
Figura 55: Fios de nylon encaixados nos attaches dos servos



4.2.9 – Criação da base do ARMIAC

Foi criada uma base para melhor apresentação e portabilidade do projeto.

Figura 56: Base do Armiac



4.2.10 – Testes elétricos e medições com multímetro

Antes de ligar o sistema completo à fonte de alimentação, foi utilizado um multímetro para medir a voltagem e garantir que o valor se encontrava correto (**6V**). Esta verificação permitiu aumentar a segurança do sistema e reduzir riscos de danos nos componentes.

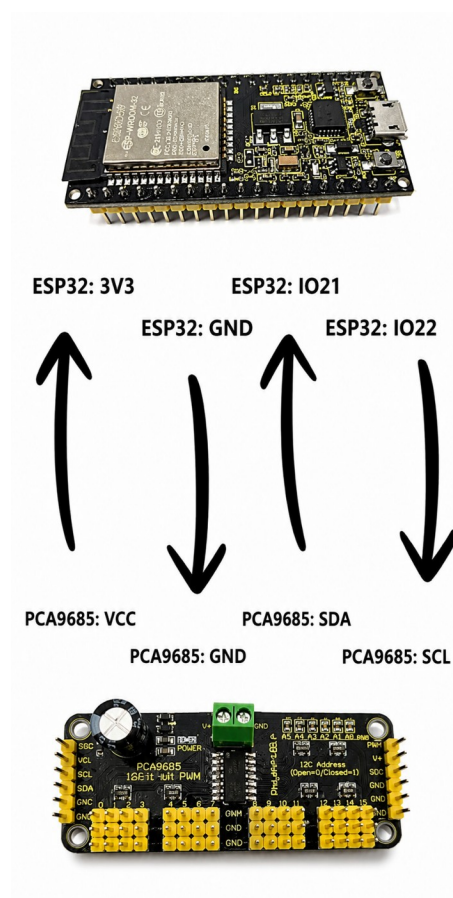
Figura 57: Medição de tensão com multímetro antes de ligar o sistema



4.2.11 – Ligações entre ESP32 e PCA9685

Nesta etapa foi realizada a ligação entre o ESP32 e o módulo PCA9685 através da comunicação I2C. O pino 3V3 do ESP32 foi ligado ao VCC do PCA9685 para alimentar a parte lógica do módulo. O GND do ESP32 foi ligado ao GND do PCA9685, garantindo uma referência comum entre os componentes. Para a comunicação, foram utilizados os pinos GPIO21 como SDA e GPIO22 como SCL. Desta forma, o ESP32 consegue enviar instruções ao PCA9685, que depois gera os sinais PWM necessários para controlar os servomotores.

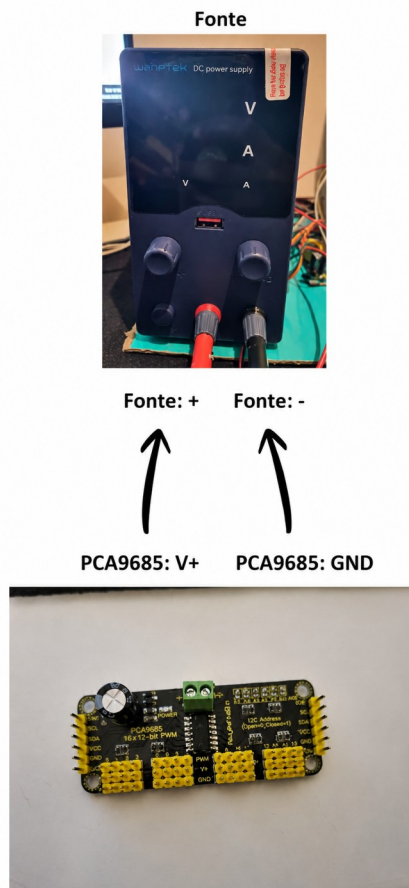
Figura 58: Esquema de ligação entre o ESP32 e o módulo PCA9685



4.2.12 – Alimentação e primeiros testes

Após a montagem mecânica e eletrônica, foi realizada a alimentação do sistema utilizando a fonte DC de 6V 10A. Foram feitos testes iniciais para verificar se todos os servos respondiam corretamente aos comandos.

Figura 59: Ligação da fonte de alimentação ao PCA9685



Demonstração em “Demonstração do funcionamento dos dedos” nos anexos, na secção 7.3

4.3 – Programação do ESP32 (Arduino IDE)

A programação do ESP32 foi feita no Arduino IDE. O microcontrolador foi configurado para criar uma rede Wi-Fi (Access Point) e receber comandos através de um servidor web.

Foram criadas rotas HTTP para permitir o controlo remoto do braço robótico através da aplicação Flutter.

4.4 – Programação da aplicação Flutter

Para o controlo do ARMIAC foi desenvolvida uma aplicação móvel em Flutter, permitindo enviar comandos HTTP ao ESP32.

A aplicação possui uma página inicial com o estado de ligação e uma página de controlo onde é possível abrir/fechar a mão e controlar os dedos individualmente.

Figura 60: Página inicial da app

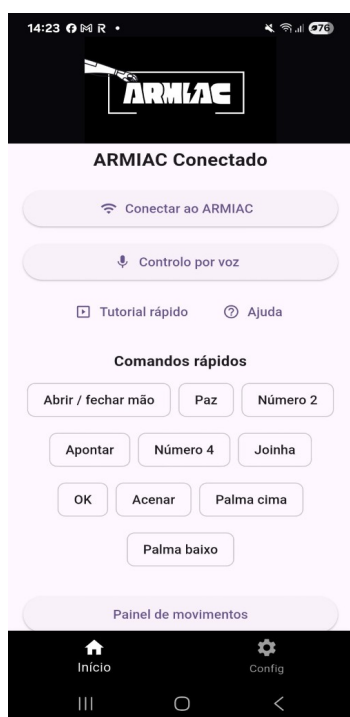


Figura 61: Slides de mini-tutorial da app - 1



Figura 62: Slides de mini-tutorial da app - 2



Figura 63: Painel de controlos na app

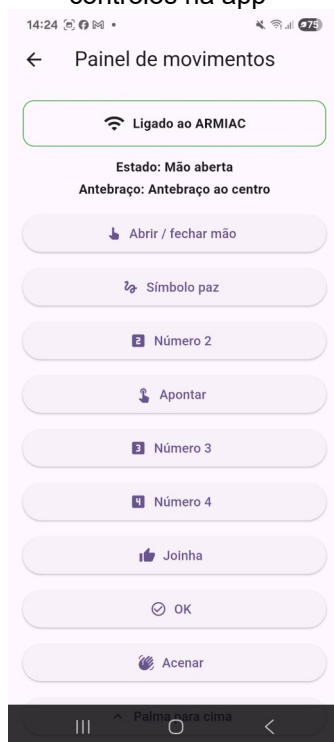


Figura 64: Lista de comandos por voz possíveis

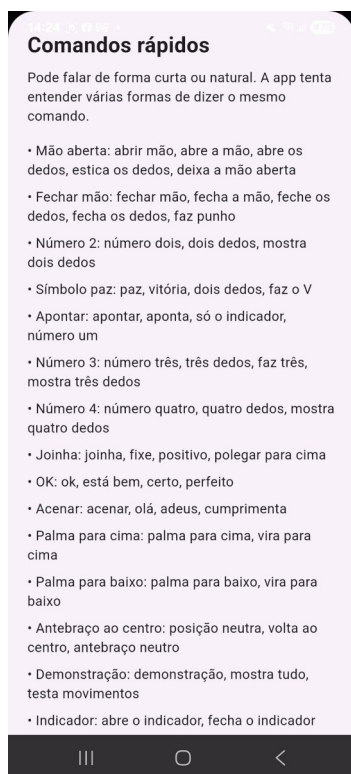
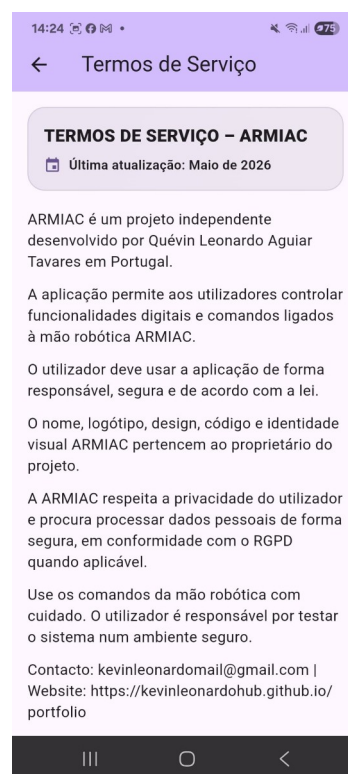


Figura 65: Página de configurações



Figura 66: Página de termos de serviço



4.4.1 – Criação do projeto Flutter

Para iniciar o desenvolvimento da aplicação móvel do ARMIAC, foi criado um projeto Flutter através do terminal. Primeiro foi necessário instalar o **Flutter SDK**, configurar as variáveis de ambiente e confirmar a instalação com o comando **flutter doctor**.

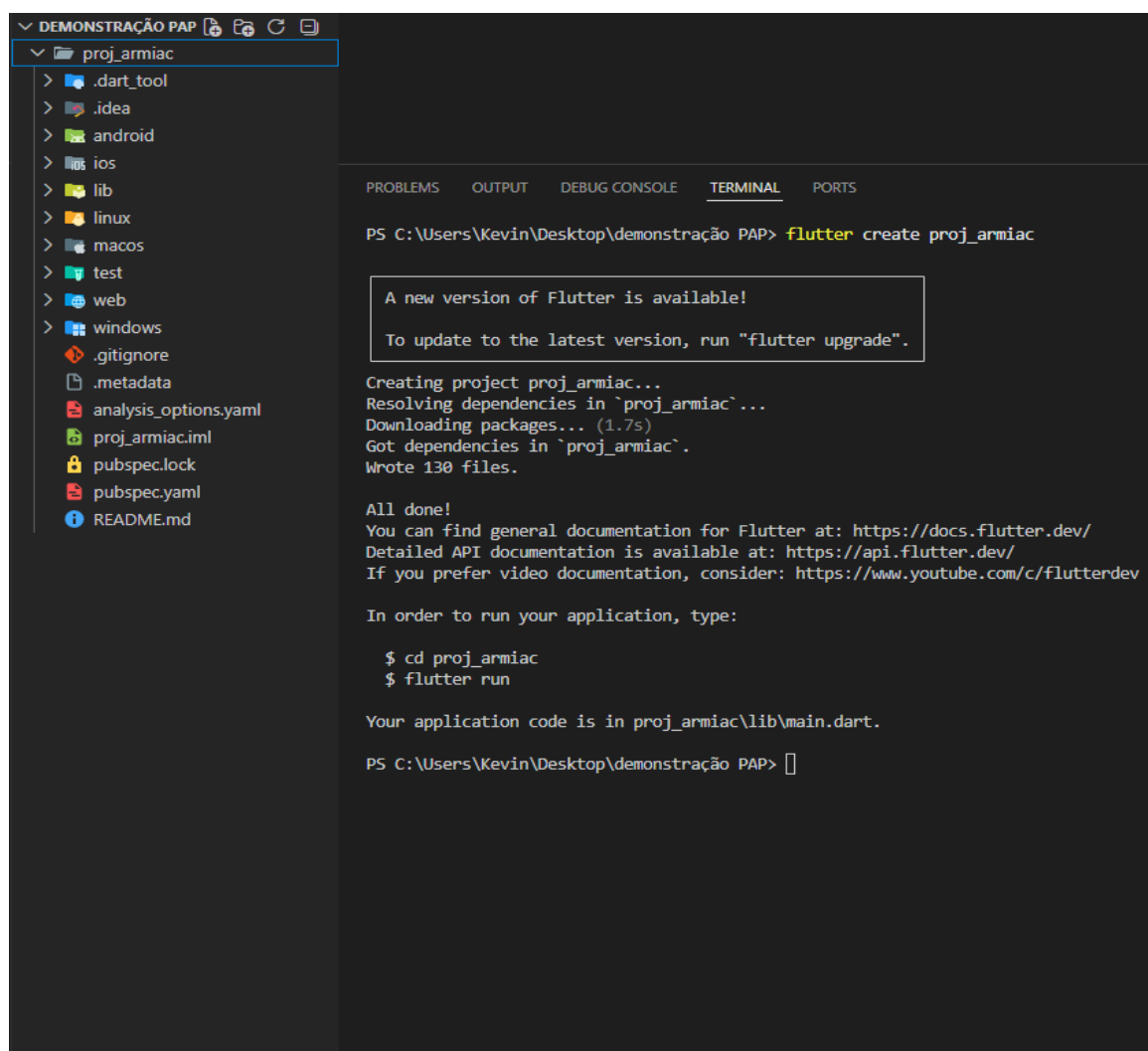
Depois de o ambiente estar configurado corretamente, foi criado o projeto com o comando:

flutter create armiac_app

Em seguida, o projeto foi aberto no Visual Studio Code, onde foram criadas as páginas, serviços e ficheiros necessários para a aplicação. Durante o desenvolvimento foram utilizados comandos como **flutter pub get**, para instalar as dependências, e **flutter run**, para testar a aplicação num dispositivo Android.

A estrutura inicial do Flutter foi depois adaptada ao projeto ARMIAC, separando o código por páginas, serviços e recursos, tornando a aplicação mais organizada e fácil de manter.

Figura 67: Criação do projeto flutter



The screenshot shows the Visual Studio Code interface. On the left, the Explorer view displays the file structure of the 'proj_armiac' project, including folders for .dart_tool, .idea, android, ios, lib, linux, macos, test, web, windows, and files like .gitignore, .metadata, analysis_options.yaml, proj_armiac.iml, pubspec.lock, pubspec.yaml, and README.md. The main editor area shows the TERMINAL output for the command 'flutter create proj_armiac'. The output indicates that a new version of Flutter is available and provides instructions to update it. It then shows the successful creation of the project, resolving dependencies, and downloading packages. Finally, it provides the commands to run the application: 'cd proj_armiac' and 'flutter run'. The application code is noted to be in 'proj_armiac\lib\main.dart'.

```
PS C:\Users\Kevin\Desktop\demonstração PAP> flutter create proj_armiac

A new version of Flutter is available!
To update to the latest version, run "flutter upgrade".

Creating project proj_armiac...
Resolving dependencies in `proj_armiac`...
Downloading packages... (1.7s)
Got dependencies in `proj_armiac`.
Wrote 130 files.

All done!
You can find general documentation for Flutter at: https://docs.flutter.dev/
Detailed API documentation is available at: https://api.flutter.dev/
If you prefer video documentation, consider: https://www.youtube.com/c/flutterdev

In order to run your application, type:

$ cd proj_armiac
$ flutter run

Your application code is in proj_armiac\lib\main.dart.
PS C:\Users\Kevin\Desktop\demonstração PAP>
```

A aplicação ARMIAC foi desenvolvida e testada apenas para Android. Embora o Flutter permita criar aplicações para Android e iOS, não foi possível gerar uma versão para iPhone, pois para isso seria necessário ter um computador com macOS, como um MacBook para compilar a aplicação. Também seria necessário um iPhone para testar a instalação, a interface, a ligação Wi-Fi e o envio de comandos para o ESP32. Como estes equipamentos não estavam disponíveis, a versão final da aplicação foi preparada **apenas para Android**.

4.4.2 – Gestão de dependências e recursos (pubspec.yaml)

O ficheiro **pubspec.yaml** é um dos ficheiros mais importantes de um projeto Flutter, pois é nele que se definem:

- as dependências externas (packages),
- a versão do SDK,
- os recursos do projeto (assets), como imagens,
- e informações básicas da aplicação (nome e versão).

No ARMIAC, o **pubspec.yaml** foi utilizado para integrar bibliotecas essenciais:

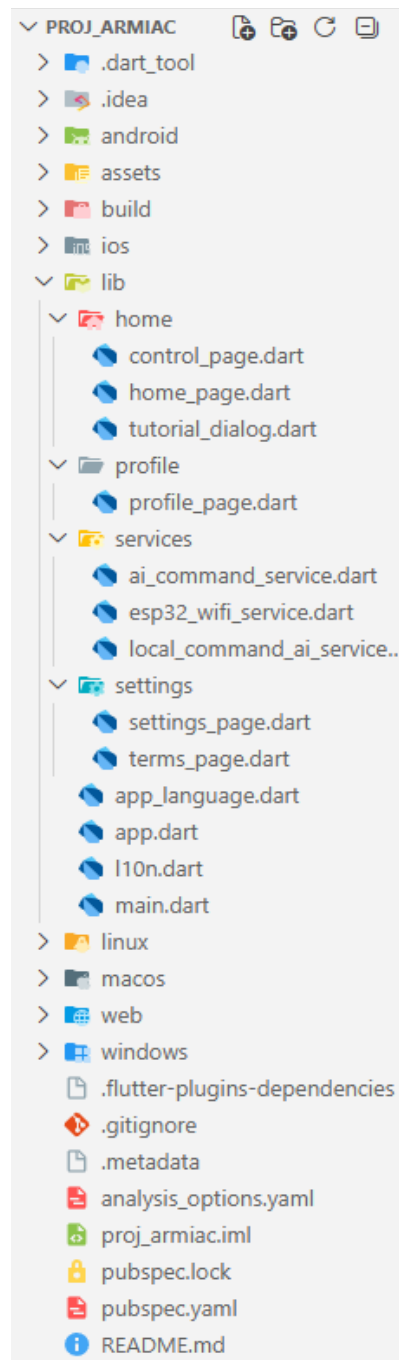
- **http** – permite realizar pedidos HTTP ao ESP32 (utilizado nos endpoints **/ping** e **/action**);
- **package_info_plus** – utilizado para obter e apresentar a versão da aplicação na área de configurações;
- **assets (imagens)** – utilizado para carregar o logótipo do projeto dentro da aplicação.

A definição correta das dependências e dos assets no **pubspec.yaml** foi fundamental para garantir que a aplicação funcionasse corretamente e que a imagem do logótipo fosse carregada no topo do aplicativo.

4.4.3 – Organização do projeto Flutter (estrutura de pastas)

Durante o desenvolvimento da aplicação ARMIAC, as pastas foram organizadas com o objetivo de manter o código mais limpo e fácil de entender.

Figura 68: Estrutura das pastas



4.4.4 – Geração do APK da aplicação

Depois de concluir o desenvolvimento da aplicação, foi gerado um ficheiro **APK** para permitir a instalação da app ARMIAC em dispositivos Android.

Para criar o APK foi utilizado o comando:

flutter build apk --release

Após a conclusão do processo, o ficheiro APK foi gerado na pasta:

build/app/outputs/flutter-apk/app-release.apk

Este ficheiro pode ser instalado num telemóvel Android, permitindo utilizar a aplicação sem necessidade de executar o projeto diretamente pelo Visual Studio Code.

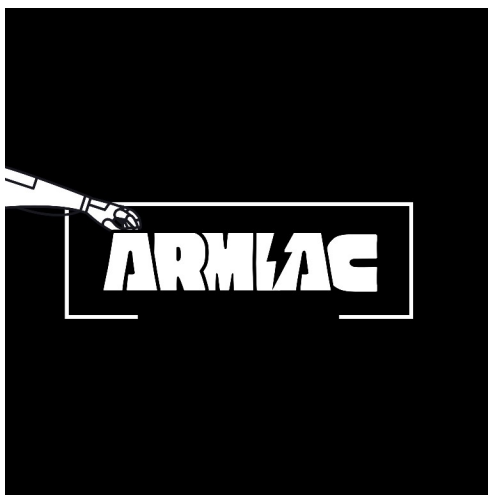
O APK final da aplicação ARMIAC encontra-se disponível nos anexos, na secção 7.3.

4.5 – Identidade Visual do Projeto (Logótipo ARMIAC)

Para dar uma identidade visual ao projeto, foi desenvolvido um logótipo com o nome ARMIAC. O logótipo foi criado utilizando o software Canva, permitindo criar uma imagem representativa do braço robótico e do nome do projeto.

O logótipo foi posteriormente integrado na aplicação Flutter, sendo apresentado no topo da interface principal e no ícone.

Figura 69: Logo ARMIAC



4.6 – Dificuldades encontradas durante o desenvolvimento

Durante o desenvolvimento do projeto ARMIAC surgiram várias dificuldades técnicas que exigiram pesquisa, testes e adaptação de soluções, tanto na parte física como na parte de software.

Uma das principais dificuldades esteve relacionada com a impressão 3D das peças. Em algumas tentativas ocorreram falhas nos modelos e erros durante a impressão, como deformações e problemas de aderência à base, o que levou à repetição de várias impressões e a ajustes nos ficheiros STL e nas configurações de slicing. Este processo aumentou o tempo de desenvolvimento, mas permitiu compreender melhor a importância da preparação correta dos modelos e das definições da impressora Elegoo Neptune 4.

Outra dificuldade encontrada foi relacionada com o acabamento das peças. Numa primeira versão, foi feita uma tentativa de imprimir as peças e depois pintá-las, mas o resultado final não ficou com a qualidade pretendida. A pintura não apresentou um acabamento uniforme e o aspeto visual não ficou satisfatório. Por esse motivo, foi necessário comprar PLA branco, criar a minha versão melhorada das peças e imprimi-las novamente, conseguindo assim um resultado mais limpo e adequado ao protótipo final.

Também surgiram dificuldades na alimentação elétrica do sistema. Tentei utilizar uma bateria, acreditando que seria mais prático e a sua potência seria suficiente para alimentar os servomotores e os restantes componentes. No entanto, durante os testes, verificou-se que a bateria não fornecia energia suficiente para garantir o funcionamento estável do sistema.

Durante os testes eletrónicos, ocorreu ainda a avaria de dois módulos PCA9685 devido à utilização de uma voltagem incorreta. Este problema atrasou o desenvolvimento, pois foi necessário substituir os componentes danificados e rever com mais cuidado as ligações elétricas, principalmente no que diz respeito à tensão correta de alimentação dos módulos e dos servomotores.

Outra dificuldade relevante foi a instalação e configuração do Flutter no ambiente de desenvolvimento. Durante este processo, ocorreram erros em que o Flutter não era reconhecido corretamente pelo sistema, devido a problemas no PATH, ou seja, na variável de ambiente. O erro mais frequente era o sistema não conseguir localizar o comando flutter, impedindo a execução de comandos no terminal. Para resolver este problema foi necessário configurar manualmente o PATH, garantindo que a pasta bin do Flutter estava corretamente adicionada às variáveis do sistema, permitindo assim que o Flutter fosse executado corretamente.

Além disso, apesar de não ter sido exatamente uma dificuldade, foi necessário aprender a linguagem Dart desde o início, uma vez que não tinha experiência prévia com esta linguagem. Foi necessário estudar a sintaxe, a estrutura de projetos Flutter e conceitos como widgets, navegação entre páginas e integração de packages. Esta aprendizagem foi essencial para conseguir desenvolver uma aplicação funcional e estável para controlar o protótipo.

Apesar destas dificuldades, todas foram ultrapassadas através de pesquisa, testes e persistência, contribuindo para uma evolução significativa das competências técnicas.

Demonstração em “Demonstração do projeto com bateria vs com fonte” nos anexos, na secção 7.3.

5 – Conclusão

Com a realização deste projeto foi possível desenvolver um protótipo funcional de uma mão robótica controlada através de uma aplicação móvel. O ARMIAC permitiu aplicar e relacionar várias áreas estudadas ao longo do curso, como programação, eletrônica, redes, desenvolvimento de aplicações, impressão 3D e resolução de problemas técnicos.

O desenvolvimento do projeto foi realizado por etapas, começando pela pesquisa e planeamento, seguindo depois para a criação e impressão das peças em 3D, montagem mecânica, instalação dos servomotores, ligação ao módulo PCA9685, programação do ESP32 e desenvolvimento da aplicação em Flutter. Ao longo deste processo foram realizados vários testes e ajustes, até se obter uma versão funcional do protótipo.

A aplicação móvel desenvolvida, na versão 1.1.7, permite controlar o ARMIAC através de ligação Wi-Fi, botões, comandos rápidos, painel de movimentos e comandos por voz. A app inclui também suporte em português e inglês, tutorial rápido, lista de comandos disponíveis e página de termos de serviço, tornando a utilização mais simples e organizada.

Uma das partes mais importantes do projeto foi a integração entre a aplicação móvel e o sistema físico. A aplicação envia comandos ao ESP32 através do protocolo HTTP; o ESP32 comunica com o PCA9685 por I2C; e o PCA9685 controla os servomotores através de sinais PWM. Esta ligação entre software e hardware foi essencial para transformar os comandos da aplicação em movimentos reais da mão robótica.

Durante o desenvolvimento surgiram algumas dificuldades, principalmente relacionadas com a alimentação elétrica, a montagem dos fios dos dedos, a calibração dos servomotores e a estabilidade dos movimentos. A tentativa de utilização de uma bateria revelou-se instável, sendo posteriormente substituída por uma fonte DC de 6V 10A, que garantiu um funcionamento mais seguro e adequado dos servomotores.

No geral, o projeto cumpriu o objetivo principal: criar uma mão robótica funcional, controlada remotamente e com diferentes formas de interação. Além disso, o ARMIAC demonstrou ser uma solução com potencial educativo, podendo ser utilizado como uma plataforma de aprendizagem para demonstrar conceitos de robótica, programação, eletrônica, comunicação Wi-Fi, impressão 3D e desenvolvimento de aplicações móveis.

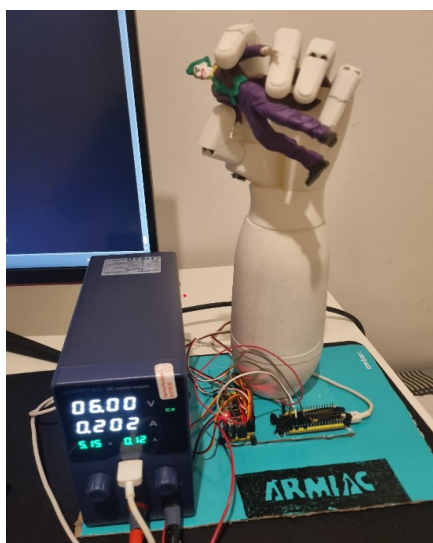
5.1 – Melhorias futuras

Apesar de o projeto ter cumprido o objetivo principal, existem algumas melhorias que poderiam ser implementadas numa versão futura do ARMIAC. Uma das melhorias seria a utilização de uma bateria mais adequada, capaz de fornecer energia suficiente aos servomotores sem causar instabilidade. Também seria possível adicionar sensores de pressão nos dedos, permitindo ao braço detetar quando está a segurar um objeto.

Outra melhoria seria aumentar a precisão dos movimentos, através de uma melhor calibração dos servomotores e de uma estrutura mecânica mais resistente. A aplicação móvel também poderia ser melhorada com novas funcionalidades, como perfis de movimentos personalizados, histórico de comandos e uma interface ainda mais intuitiva. No futuro, também poderia ser desenvolvida uma versão da aplicação para iOS, caso existissem os equipamentos necessários para compilar e testar a aplicação em dispositivos Apple. Além disso, o ARMIAC poderia ser evoluído para incluir controlo por Bluetooth, integração com sensores ou até movimentos automáticos programados.

Melhoria futura	Objetivo
Bateria mais adequada	Tornar o projeto mais portátil
Sensores de pressão nos dedos	Detetar objetos e força de aperto
Melhor calibração dos servos	Movimentos mais suaves e precisos
Estrutura mecânica reforçada	Aumentar resistência e durabilidade
Versão iOS da aplicação	Permitir uso em iPhone
Perfis de movimentos personalizados	Guardar gestos criados pelo utilizador
Melhor organização dos fios	Melhorar estética e manutenção
Controlo por Bluetooth	Criar alternativa ao Wi-Fi

Figura 70: Projeto Finalizado



6 – Webgrafia

- Documentação Arduino – <https://www.arduino.cc>
- Documentação Espressif ESP32 – <https://www.espressif.com>
- Documentação ESP32 Arduino – <https://docs.espressif.com>
- Documentação Adafruit PCA9685 – <https://learn.adafruit.com>
- Biblioteca Adafruit PWM Servo Driver – <https://github.com/adafruit/Adafruit-PWM-Servo-Driver-Library>
- Documentação PCA9685 – <https://www.nxp.com>
- Documentação Flutter – <https://docs.flutter.dev>
- Documentação Dart – <https://dart.dev>
- Package http – <https://pub.dev/packages/http>
- Package speech_to_text – https://pub.dev/packages/speech_to_text
- Package flutter_tts – https://pub.dev/packages/flutter_tts
- Package shared_preferences – https://pub.dev/packages/shared_preferences
- Package package_info_plus – https://pub.dev/packages/package_info_plus
- LibreOffice – <https://www.libreoffice.org>
- Documentação Visual Studio Code – <https://code.visualstudio.com/docs>
- Documentação Android Studio – <https://developer.android.com/studio>
- Tinkercad – <https://www.tinkercad.com>
- Ultimaker Cura – <https://ultimaker.com/software/ultimaker-cura>
- Elegoo – <https://www.elegoo.com>
- Canva – <https://www.canva.com>
- Servo MZ996R – <https://www.aliexpress.com/s/wiki-ssr/article/mz996-servo>
- Regulamento Geral sobre a Proteção de Dados – <https://eur-lex.europa.eu>

- Comissão Nacional de Proteção de Dados – <https://www.cnpd.pt>
- Projeto base de inspiração - <https://www.thingiverse.com/thing:2269115>

7 – Anexos

7.1 - Código

Código ESP32(C++):

```
#include <Wire.h>
#include <WiFi.h>
#include <WebServer.h>
#include <Adafruit_PWMServoDriver.h>

Adafruit_PWMServoDriver pca = Adafruit_PWMServoDriver(0x40);
WebServer server(80);

const char* ssid = "ARMIAC";
const char* pass = "12345678";

#define SERVO_MIN 110
#define SERVO_MAX 500

#define CH_POLEGAR_DENTRO 0
#define CH_POLEGAR_FLEX 1
#define CH_INDICADOR 2
#define CH_MEDIO 3
#define CH_RINGPINKY 4
#define CH_ANTEBRACO 5

#define ANG_INDICADOR_ABERTO 180
#define ANG_INDICADOR_FECHADO 0
#define ANG_POLEGAR_DENTRO_ABERTO 0
#define ANG_POLEGAR_DENTRO_FECHADO 140

String lastAction = "open_hand";

// Converte o ângulo do servo para o valor de pulso usado pelo controlador PCA9685.
uint16_t anguloParaPulso(int angulo) {
    return map(angulo, 0, 180, SERVO_MIN, SERVO_MAX);
}

// Define o ângulo de um servo num determinado canal.
void setServoAng(int ch, int ang) {
    ang = constrain(ang, 0, 180);
    pca.setPWM(ch, 0, anguloParaPulso(ang));
}

// Devolve o ângulo invertido conforme o dedo esteja fechado ou aberto.
int angInvertido(int fechado) {
```

```

    return fechado ? 0 : 180;
}

// Controla o dedo indicador.
void setIndicador(int fechado) {
    setServoAng(CH_INDICADOR, fechado ? ANG_INDICADOR_FECHADO :
ANG_INDICADOR_ABERTO);
}

// Controla o dedo médio.
void setMedio(int fechado) {
    setServoAng(CH_MEDIO, angInvertido(fechado));
}

// Controla o anelar e o mindinho.
void setRingPinky(int fechado) {
    setServoAng(CH_RINGPINKY, angInvertido(fechado));
}

// Controla a flexão do polegar.
void setPolegarFlex(int fechado) {
    setServoAng(CH_POLEGAR_FLEX, angInvertido(fechado));
}

// Controla o movimento do polegar para dentro da mão.
void setPolegarDentro(int fechado) {
    int ang = fechado ? ANG_POLEGAR_DENTRO_FECHADO :
ANG_POLEGAR_DENTRO_ABERTO;
    setServoAng(CH_POLEGAR_DENTRO, ang);
}

// Abre todos os dedos da mão.
void actionOpenHand() {
    setIndicador(0);
    setMedio(0);
    setRingPinky(0);
    setPolegarFlex(0);
    setPolegarDentro(0);
}

// Fecha todos os dedos da mão.
void actionCloseHand() {
    setIndicador(1);
    setMedio(1);
    setRingPinky(1);
    setPolegarFlex(1);
    setPolegarDentro(1);
}

```



```
}

// Faz o gesto de paz.
void actionPeace() {
    setIndicador(0);
    setMedio(0);
    setRingPinky(1);
    setPolegarFlex(1);
    setPolegarDentro(1);
}

// Faz o gesto do número dois.
void actionTwo() {
    actionPeace();
}

// Faz o gesto de apontar.
void actionPoint() {
    setIndicador(0);
    setMedio(1);
    setRingPinky(1);
    setPolegarFlex(1);
    setPolegarDentro(1);
}

// Faz o gesto do número três.
void actionThree() {
    setIndicador(0);
    setMedio(0);
    setRingPinky(1);
    setPolegarFlex(0);
    setPolegarDentro(0);
}

// Faz o gesto do número quatro.
void actionFour() {
    setIndicador(0);
    setMedio(0);
    setRingPinky(0);
    setPolegarFlex(1);
    setPolegarDentro(1);
}

// Faz o gesto de polegar para cima.
void actionThumbsUp() {
    setIndicador(1);
    setMedio(1);
}
```

```

    setRingPinky(1);
    setPolegarFlex(0);
    setPolegarDentro(0);
}

// Faz uma aproximação do gesto OK.
void actionOk() {
    setIndicador(1);
    setMedio(0);
    setRingPinky(0);
    setPolegarFlex(1);
    setPolegarDentro(1);
}

// Roda o antebraço para uma posição com a palma para cima.
void actionPalmUp() {
    setServoAng(CH_ANTEBRACO, 35);
}

// Roda o antebraço para uma posição com a palma para baixo.
void actionPalmDown() {
    setServoAng(CH_ANTEBRACO, 145);
}

// Coloca o antebraço numa posição neutra.
void actionForearmNeutral() {
    setServoAng(CH_ANTEBRACO, 90);
}

// Faz um pequeno movimento de aceno com o antebraço.
void actionWave() {
    for (int repeticao = 0; repeticao < 2; repeticao++) {
        actionPalmUp();
        delay(250);
        actionPalmDown();
        delay(250);
    }

    actionForearmNeutral();
}

// Executa uma sequência de demonstração com vários gestos.
void actionDemo() {
    actionOpenHand();
    delay(500);

    actionCloseHand();
}

```

```
delay(500);

actionOpenHand();
delay(500);

actionPeace();
delay(700);

actionPoint();
delay(700);

actionThree();
delay(700);

actionThumbsUp();
delay(700);

actionOk();
delay(700);

actionWave();
delay(300);

actionOpenHand();
}

// Fecha apenas o indicador.
void actionCloseIndex() {
    setIndicador(1);
}

// Fecha apenas o dedo médio.
void actionCloseMiddle() {
    setMedio(1);
}

// Fecha o anelar e o mindinho.
void actionCloseRingPinky() {
    setRingPinky(1);
}

// Fecha o polegar.
void actionCloseThumb() {
    setPolegarFlex(1);
    setPolegarDentro(1);
}
```

```

// Abre apenas o indicador.
void actionOpenIndex() {
    setIndicador(0);
}

// Abre apenas o dedo médio.
void actionOpenMiddle() {
    setMedio(0);
}

// Abre o anelar e o mindinho.
void actionOpenRingPinky() {
    setRingPinky(0);
}

// Abre o polegar.
void actionOpenThumb() {
    setPolegarFlex(0);
    setPolegarDentro(0);
}

// Responde ao pedido de teste da ligação.
void handlePing() {
    server.send(200, "text/plain", "OK");
}

// Envia o estado atual da mão em formato JSON.
void handleStatus() {
    String json = "{\"status\":\"ok\",\"lastAction\":\"" + lastAction + "\",\"ip\":\"" +
WiFi.softAPIP().toString() + "\"}";
    server.send(200, "application/json", json);
}

// Permite testar manualmente um servo através do navegador.
void handleCalibrate() {
    if (!server.hasArg("servo") || !server.hasArg("angle")) {
        server.send(400, "text/plain", "Use: /calibrate?servo=2&angle=90");
        return;
    }

    int servo = server.arg("servo").toInt();
    int angle = server.arg("angle").toInt();

    if (servo < 0 || servo > 15 || angle < 0 || angle > 180) {
        server.send(400, "text/plain", "servo deve ser 0-15 e angle deve ser 0-180");
        return;
    }
}

```

```

    setServoAng(servo, angle);
    lastAction = "calibrate servo " + String(servo) + " angle " + String(angle);

server.send(200, "text/plain", "OK");
}

// Recebe o nome da ação pela URL e executa o gesto correspondente.
void handleAction() {
    String a = server.arg("a");

    if (a == "close_hand") actionCloseHand();
    else if (a == "peace") actionPeace();
    else if (a == "two") actionTwo();
    else if (a == "point") actionPoint();
    else if (a == "three") actionThree();
    else if (a == "four") actionFour();
    else if (a == "thumbs_up") actionThumbsUp();
    else if (a == "ok") actionOk();
    else if (a == "wave") actionWave();
    else if (a == "palm_up") actionPalmUp();
    else if (a == "palm_down") actionPalmDown();
    else if (a == "forearm_neutral") actionForearmNeutral();
    else if (a == "demo") actionDemo();
    else if (a == "close_index") actionCloseIndex();
    else if (a == "close_middle") actionCloseMiddle();
    else if (a == "close_ringpinky") actionCloseRingPinky();
    else if (a == "close_thumb") actionCloseThumb();
    else if (a == "open_hand") actionOpenHand();
    else if (a == "open_index") actionOpenIndex();
    else if (a == "open_middle") actionOpenMiddle();
    else if (a == "open_ringpinky") actionOpenRingPinky();
    else if (a == "open_thumb") actionOpenThumb();
    else {
        server.send(400, "text/plain", "Acao invalida");
        return;
    }

    lastAction = a;
    server.send(200, "text/plain", "OK");
}

// Configura os servos, cria a rede Wi-Fi e inicia o servidor.
void setup() {
    Serial.begin(115200);

    Wire.begin(21, 22);

```

```

pca.begin();
pca.setPWMPFreq(50);
delay(10);

actionOpenHand();
actionForearmNeutral();

WiFi.mode(WIFI_AP);
WiFi.softAP(ssid, pass);

Serial.print("AP: ");
Serial.println(ssid);

Serial.print("IP: ");
Serial.println(WiFi.softAPIP());

server.on("/ping", handlePing);
server.on("/status", handleStatus);
server.on("/calibrate", handleCalibrate);
server.on("/action", handleAction);

server.begin();

Serial.println("READY");
}

// Mantém o servidor ativo para receber pedidos.
void loop() {
  server.handleClient();
}

```

Código Flutter(Dart):

Diretório – proj_armiac/lib/main.dart

```

import 'package:flutter/material.dart';
import 'app.dart';

/// Ponto de entrada da aplicação Flutter.
void main() {
  runApp(const MyApp());
}

```

Diretório – proj_armiac/lib/app.dart

```
import 'package:flutter/material.dart';
```

```
import 'app_language.dart';  
import 'home/home_page.dart';  
import 'l10n.dart';  
import 'settings/settings_page.dart';
```

/// Widget principal da aplicação. Fica responsável por montar o MaterialApp.

```
class MyApp extends StatelessWidget {  
  const MyApp({super.key});
```

```
  @override
```

/// Reconstroi a aplicação quando as definições globais mudam.

```
  Widget build(BuildContext context) {
```

```
    return AnimatedBuilder(  
      animation: appSettings,  
      builder: (context, _) {  
        return MaterialApp(  
          debugShowCheckedModeBanner: false,  
          title: 'ARMIAC',  
          locale: appSettings.language.locale,  
          theme: ThemeData(  
            colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepPurple),  
            useMaterial3: true,  
          ),  
          home: const MyHomePage(),  
        );  
      },  
    );  
  }  
}
```

/// Página base com navegação entre o início e as configurações.

```
class MyHomePage extends StatefulWidget {  
  const MyHomePage({super.key});
```

```
  @override
```

```
  State<MyHomePage> createState() => _MyHomePageState();  
}
```

/// Guarda o separador selecionado e atualiza o ecrã quando o utilizador muda de página.

```
class _MyHomePageState extends State<MyHomePage> {  
  int _selectedIndex = 0;
```

/// Atualiza o índice da barra inferior.

```
  void _onItemTapped(int index) => setState(() => _selectedIndex = index);
```



```

@override
/// Reconstrói a aplicação quando as definições globais mudam.
Widget build(BuildContext context) {
  return AnimatedBuilder(
    animation: appSettings,
    builder: (context, _) {
      final l10n = L10n(appSettings.language);
      final pages = const [HomePage(), SettingsPage()];

      return Scaffold(
        appBar: AppBar(
          backgroundColor: Colors.black,
          centerTitle: true,
          toolbarHeight: 120,
          title: SizedBox(
            height: 200,
            child: Image.asset(
              'assets/images/Armiac_LOGO.png',
              fit: BoxFit.contain,
            ),
          ),
        ),
        body: pages[_selectedIndex],
        bottomNavigationBar: BottomNavigationBar(
          currentIndex: _selectedIndex,
          onTap: _onItemTapped,
          backgroundColor: Colors.black,
          selectedItemColor: Colors.white,
          unselectedItemColor: Colors.grey,
          items: [
            BottomNavigationBarItem(
              icon: const Icon(Icons.home),
              label: l10n.home,
            ),
            BottomNavigationBarItem(
              icon: const Icon(Icons.settings),
              label: l10n.configShort,
            ),
          ],
        ),
      );
    },
  );
}

```

Diretório – proj_armiac/lib/app_language.dart

```
import 'package:flutter/material.dart';

/// Idiomas suportados pela aplicação.
enum AppLanguage {
  portuguese,
  english,
}

/// Junta ao idioma os códigos usados pela interface, voz e leitura de texto.
extension AppLanguageInfo on AppLanguage {
  String get code => this == AppLanguage.portuguese ? 'pt_BR' : 'en_US';
  String get speechLocale => code;
  String get ttsLocale => this == AppLanguage.portuguese ? 'pt-PT' : 'en-US';
  Locale get locale => this == AppLanguage.portuguese ? const Locale('pt', 'PT') : const
Locale('en', 'US');
  String get label => this == AppLanguage.portuguese ? 'Português' : 'English';
}

/// Controlador simples para guardar definições globais e avisar a interface quando
mudam.
class AppSettingsController extends ChangeNotifier {
  AppLanguage _language = AppLanguage.portuguese;

  AppLanguage get language => _language;

  /// Troca o idioma e obriga os widgets dependentes a atualizar.
  void setLanguage(AppLanguage value) {
    if (_language == value) return;
    _language = value;
    notifyListeners();
  }
}

/// Instância global usada pelas páginas para ler e alterar o idioma atual.
final appSettings = AppSettingsController();
```

Diretório – proj_armiac/lib/l10n.dart

```
import 'app_language.dart';

/// Centraliza todos os textos da aplicação em português e inglês.
class L10n {
  const L10n(this.language);
```

```

final AppLanguage language;
bool get isPt => language == AppLanguage.portuguese;

String get home => isPt ? 'Início' : 'Home';
String get settings => isPt ? 'Configurações' : 'Settings';
String get configShort => isPt ? 'Config' : 'Settings';
String get appVersion => isPt ? 'Versão da app' : 'App version';
String get serviceTerms =>
  isPt ? 'Termos de serviço' : 'Terms of service';
String get appName => 'ARMIAC';
String get appSubtitle =>
  isPt ? 'Mão robótica inteligente' : 'Intelligent robotic hand';
String get languageLabel => isPt
  ? 'Idioma da app e comandos de voz'
  : 'App and voice command language';
String get portuguese => 'Português';
String get english => 'English';
String get languageHelp => isPt
  ? 'Ao escolher português, a app fala, escuta e entende comandos em português.'
  : 'When English is selected, the app speaks, listens, and understands English
commands.';
String get microphonePermission =>
  isPt ? 'Permissão do microfone' : 'Microphone permission';
String get microphonePermissionBody => isPt
  ? 'Ative a permissão do microfone nas definições do telemóvel para usar comandos
por voz.'
  : 'Enable microphone permission in your phone settings to use voice commands.';
String get armiacConnection =>
  isPt ? 'Ligação ao ARMIAC' : 'ARMIAC connection';
String get armiacConnectionBody => isPt
  ? 'Ligue o ARMIAC à corrente, conecte o telemóvel ao Wi-Fi “ARMIAC” e depois toque
em “Conectar ao ARMIAC”.'
  : 'Plug the ARMIAC into power, connect your phone to the “ARMIAC” Wi-Fi, and then
tap “Connect to ARMIAC”.';
String get connected => isPt ? 'ARMIAC Conectado' : 'ARMIAC Connected';
String get disconnected =>
  isPt ? 'ARMIAC Desconectado' : 'ARMIAC Disconnected';
String get connecting => isPt ? 'Conectando...' : 'Connecting...';
String get connectArmiac =>
  isPt ? 'Conectar ao ARMIAC' : 'Connect to ARMIAC';
String get stopVoice => isPt ? 'Parar voz' : 'Stop voice';
String get voiceControl => isPt ? 'Controlo por voz' : 'Voice control';
String get voice => isPt ? 'Voz' : 'Voice';
String get manualControl =>
  isPt ? 'Painel de movimentos' : 'Movement panel';
String get quickCommands => isPt ? 'Comandos rápidos' : 'Quick commands';

```

```

String get tutorial => isPt ? 'Tutorial rápido' : 'Quick tutorial';
String get tutorialTitle => isPt ? 'Como usar o ARMIAC' : 'How to use ARMIAC';
String get tutorialStep1Title => isPt ? '1. Ligar o ARMIAC' : '1. Power on ARMIAC';
String get tutorialStep1Body => isPt
    ? 'Ligue o ARMIAC à corrente para o deixar pronto a funcionar.'
    : 'Plug the ARMIAC into power so it is ready to work.';
String get tutorialStep2Title =>
    isPt ? '2. Conectar e usar' : '2. Connect and use';
String get tutorialStep2Body => isPt
    ? 'Conecte-se à rede Wi-Fi “ARMIAC” e toque no botão “Conectar ao ARMIAC” na
app.'
    : 'Connect to the “ARMIAC” Wi-Fi network and tap “Connect to ARMIAC” in the app.';
String get previous => isPt ? 'Anterior' : 'Back';
String get next => isPt ? 'Seguinte' : 'Next';
String get finish => isPt ? 'Fechar' : 'Close';
String get connectedSnack =>
    isPt ? 'Conectado ao ARMIAC' : 'Connected to ARMIAC';
String get connectFirst => isPt
    ? 'Conecte-se primeiro ao Wi-Fi do ARMIAC.'
    : 'Connect to the ARMIAC Wi-Fi first.';
String get micUnavailable => isPt
    ? 'Microfone indisponível ou sem permissão.'
    : 'Microphone unavailable or permission denied.';
String get listening => isPt ? 'Estou a ouvir...' : 'Listening...';
String get speechNetworkError => isPt
    ? 'Internet desativada. Para usar voz em português sem internet, instale Português no
reconhecimento de voz do telemóvel.'
    : 'Internet disabled. To use voice commands without internet, install the selected
language in your phone voice recognition settings.';
String get speechLanguageError => isPt
    ? 'O português offline deste telemóvel está como Português (Brasil). A app vai tentar
usar pt_BR.'
    : 'English is not available in this phone voice recognition. Enable or install English in
the voice settings.';
String micError(String error) =>
    isPt ? 'Erro no microfone: $error' : 'Microphone error: $error';
String error(Object e) => isPt ? 'Erro: $e' : 'Error: $e';
String get actionFailed => isPt
    ? 'Falha ao enviar comando ao ARMIAC.'
    : 'Failed to send command to ARMIAC.';
String commandUnderstood(String spoken) =>
    isPt ? 'Comando entendido: $spoken' : 'Command understood: $spoken';
String get didNotUnderstand =>
    isPt ? 'Não entendi o comando' : 'I did not understand the command';
String get didNotUnderstandText => isPt
    ? 'Não entendi. Abra a ajuda para ver comandos disponíveis.'
    : 'I did not understand. Open help to see available commands.';

```

```

String get help => isPt ? 'Ajuda' : 'Help';
String get commandsHelp => isPt ? 'Comandos rápidos' : 'Quick commands';
String get commandsHelpIntro => isPt
    ? 'Pode falar de forma curta ou natural. A app tenta entender várias formas de dizer o
    mesmo comando.'
    : 'You can speak in a short or natural way. The app tries to understand many ways of
    saying the same command.';
List<String> get commandHelpItems => isPt
    ? [
        'Mão aberta: abrir mão, abre a mão, abre os dedos, estica os dedos, deixa a mão
        aberta',
        'Fechar mão: fechar mão, fecha a mão, feche os dedos, fecha os dedos, faz punho',
        'Número 2: número dois, dois dedos, mostra dois dedos',
        'Símbolo paz: paz, vitória, dois dedos, faz o V',
        'Apontar: apontar, aponta, só o indicador, número um',
        'Número 3: número três, três dedos, faz três, mostra três dedos',
        'Número 4: número quatro, quatro dedos, mostra quatro dedos',
        'Joinha: joinha, fixe, positivo, polegar para cima',
        'OK: ok, está bem, certo, perfeito',
        'Acenar: acenar, olá, adeus, cumprimenta',
        'Palma para cima: palma para cima, vira para cima',
        'Palma para baixo: palma para baixo, vira para baixo',
        'Antebraço ao centro: posição neutra, volta ao centro, antebraço neutro',
        'Demonstração: demonstração, mostra tudo, testa movimentos',
        'Indicador: abre o indicador, fecha o indicador',
        'Dedo médio: abre o dedo médio, fecha o dedo médio',
        'Anelar + mindinho: abre o anelar e o mindinho, fecha o anelar e o mindinho',
        'Polegar: abre o polegar, fecha o polegar',
    ]
    : [
        'Open hand: open hand, open the hand, open fingers, stretch fingers',
        'Close hand: close hand, close the fingers, make a fist, squeeze hand',
        'Number 2: number two, two fingers, show two fingers',
        'Peace sign: peace, victory, two fingers, make a V',
        'Point: point, only index, number one, point forward',
        'Number 3: number three, three fingers, show three',
        'Number 4: number four, four fingers, show four fingers',
        'Thumbs up: thumbs up, like, positive sign',
        'OK: ok, okay, perfect',
        'Wave: wave, hello, bye, greet',
        'Palm up: palm up, face up, rotate up',
        'Palm down: palm down, face down, rotate down',
        'Forearm center: neutral forearm, return to center',
        'Demo: demo, demonstration, show everything',
        'Index finger: open index, close index',
        'Middle finger: open middle finger, close middle finger',
        'Ring + pinky: open ring and pinky, close ring and pinky',
    ]

```

```

    'Thumb: open thumb, close thumb',
];

String get open => isPt ? 'Abrir' : 'Open';
String get close => isPt ? 'Fechar' : 'Close';
String get peace => isPt ? 'Paz' : 'Peace';
String get point => isPt ? 'Apontar' : 'Point';
String get thumbsUp => isPt ? 'Joinha' : 'Thumbs up';
String get wave => isPt ? 'Acenar' : 'Wave';
String get palmUpShort => isPt ? 'Palma cima' : 'Palm up';
String get palmDownShort => isPt ? 'Palma baixo' : 'Palm down';
String get handControl =>
    isPt ? 'Painel de movimentos' : 'Movement panel';
String get connectedToArmiac =>
    isPt ? 'Ligado ao ARMIAC' : 'Connected to ARMIAC';
String get notConnectedToArmiac =>
    isPt ? 'Não ligado ao ARMIAC' : 'Not connected to ARMIAC';
String get status => isPt ? 'Estado' : 'Status';
String get forearm => isPt ? 'Antebraço' : 'Forearm';
String get openCloseHand =>
    isPt ? 'Abrir / fechar mão' : 'Open / close hand';
String get closeHand => isPt ? 'Fechar mão' : 'Close hand';
String get peaceSign => isPt ? 'Símbolo paz' : 'Peace sign';
String get numberTwo => isPt ? 'Número 2' : 'Number 2';
String get numberThree => isPt ? 'Número 3' : 'Number 3';
String get numberFour => isPt ? 'Número 4' : 'Number 4';
String get okSign => 'OK';
String get palmUp => isPt ? 'Palma para cima' : 'Palm up';
String get palmDown => isPt ? 'Palma para baixo' : 'Palm down';
String get forearmNeutral =>
    isPt ? 'Antebraço ao centro' : 'Forearm center';
String get automaticDemo =>
    isPt ? 'Demonstração automática' : 'Automatic demo';
String get indexFinger => isPt ? 'Dedo indicador' : 'Index finger';
String get middleFinger => isPt ? 'Dedo médio' : 'Middle finger';
String get ringPinky =>
    isPt ? 'Dedo anelar + mindinho' : 'Ring + pinky fingers';
String get thumb => isPt ? 'Dedão' : 'Thumb';

String get openHandState => isPt ? 'Mão aberta' : 'Open hand';
String get closedFistState => isPt ? 'Punho fechado' : 'Closed fist';
String get demoState => isPt ? 'Demonstração' : 'Demo';
String get palmUpState => isPt ? 'Palma para cima' : 'Palm up';
String get palmDownState => isPt ? 'Palma para baixo' : 'Palm down';
String get forearmCenterState =>
    isPt ? 'Antebraço ao centro' : 'Forearm center';
String get indexClosed => isPt ? 'Indicador fechado' : 'Index closed';

```

```

String get indexOpen => isPt ? 'Indicador aberto' : 'Index open';
String get middleClosed =>
    isPt ? 'Dedo médio fechado' : 'Middle finger closed';
String get middleOpen => isPt ? 'Dedo médio aberto' : 'Middle finger open';
String get ringPinkyClosed =>
    isPt ? 'Anelar + mindinho fechados' : 'Ring + pinky closed';
String get ringPinkyOpen =>
    isPt ? 'Anelar + mindinho abertos' : 'Ring + pinky open';
String get thumbClosed => isPt ? 'Polegar fechado' : 'Thumb closed';
String get thumbOpen => isPt ? 'Polegar aberto' : 'Thumb open';

/// Devolve a frase falada depois de um comando ser executado.
String speechFor(
    String command, {
    bool indexClosed = false,
    bool middleClosed = false,
    bool ringPinkyClosed = false,
    bool thumbClosed = false,
}) {
    switch (command) {
        case 'open_hand':
            return isPt ? 'vou abrir a mão' : 'opening the hand';
        case 'close_hand':
            return isPt ? 'vou fechar a mão' : 'closing the hand';
        case 'peace':
            return isPt ? 'vou fazer o gesto de paz' : 'making the peace sign';
        case 'two':
            return isPt ? 'vou mostrar o número dois' : 'showing number two';
        case 'point':
            return isPt
                ? 'vou apontar com o indicador'
                : 'pointing with the index finger';
        case 'three':
            return isPt ? 'vou mostrar o número três' : 'showing number three';
        case 'four':
            return isPt ? 'vou mostrar o número quatro' : 'showing number four';
        case 'thumbs_up':
            return isPt ? 'vou fazer joinha' : 'making a thumbs up';
        case 'ok':
            return isPt ? 'vou fazer o gesto ok' : 'making the OK sign';
        case 'wave':
            return isPt ? 'vou acenar' : 'waving';
        case 'palm_up':
            return isPt ? 'vou virar a palma para cima' : 'turning the palm up';
        case 'palm_down':
            return isPt ? 'vou virar a palma para baixo' : 'turning the palm down';
        case 'forearm_neutral':
    
```

```

return isPt
  ? 'vou pôr o antebraço ao centro'
  : 'moving the forearm to the center';
case 'demo':
  return isPt ? 'vou fazer a demonstração' : 'starting the demonstration';
case 'open_index':
  return isPt ? 'vou abrir o indicador' : 'opening the index finger';
case 'close_index':
  return isPt ? 'vou fechar o indicador' : 'closing the index finger';
case 'open_middle':
  return isPt ? 'vou abrir o dedo médio' : 'opening the middle finger';
case 'close_middle':
  return isPt ? 'vou fechar o dedo médio' : 'closing the middle finger';
case 'open_ringpinky':
  return isPt
    ? 'vou abrir o anelar e o mindinho'
    : 'opening the ring and pinky fingers';
case 'close_ringpinky':
  return isPt
    ? 'vou fechar o anelar e o mindinho'
    : 'closing the ring and pinky fingers';
case 'open_thumb':
  return isPt ? 'vou abrir o polegar' : 'opening the thumb';
case 'close_thumb':
  return isPt ? 'vou fechar o polegar' : 'closing the thumb';
case 'toggle_index':
  return isPt
    ? (indexClosed ? 'vou abrir o indicador' : 'vou fechar o indicador')
    : (indexClosed
      ? 'opening the index finger'
      : 'closing the index finger');
case 'toggle_middle':
  return isPt
    ? (middleClosed
      ? 'vou abrir o dedo médio'
      : 'vou fechar o dedo médio')
    : (middleClosed
      ? 'opening the middle finger'
      : 'closing the middle finger');
case 'toggle_ring_pinky':
  return isPt
    ? (ringPinkyClosed
      ? 'vou abrir o anelar e o mindinho'
      : 'vou fechar o anelar e o mindinho')
    : (ringPinkyClosed
      ? 'opening the ring and pinky fingers'
      : 'closing the ring and pinky fingers');

```



```

case 'toggle_thumb':
    return isPt
        ? (thumbClosed ? 'vou abrir o polegar' : 'vou fechar o polegar')
        : (thumbClosed ? 'opening the thumb' : 'closing the thumb');
}
return "";
}

/// Textos usados na página de termos de serviço.
String get termsTitle => isPt ? 'Termos de Serviço' : 'Terms of Service';
String get termsHeader =>
    isPt ? 'TERMOS DE SERVIÇO – ARMIAC' : 'TERMS OF SERVICE – ARMIAC';
String get lastUpdate => isPt
    ? 'Última atualização: Maio de 2026'
    : 'Last updated: May 2026';
List<String> get termsParagraphs => isPt
    ? [
        'ARMiAC é um projeto independente desenvolvido por Quévin Leonardo Aguiar Tavares em Portugal.',
        'A aplicação permite aos utilizadores controlar funcionalidades digitais e comandos ligados à mão robótica ARMIAC.',
        'O utilizador deve usar a aplicação de forma responsável, segura e de acordo com a lei.',
        'O nome, logótipo, design, código e identidade visual ARMIAC pertencem ao proprietário do projeto.',
        'A ARMIAC respeita a privacidade do utilizador e procura processar dados pessoais de forma segura, em conformidade com o RGPD quando aplicável.',
        'Use os comandos da mão robótica com cuidado. O utilizador é responsável por testar o sistema num ambiente seguro.',
        'Contacto: kevinleonardomail@gmail.com | Website: https://kevinleonardohub.github.io/portfolio',
    ]
    : [
        'ARMiAC is an independent project developed by Quévin Leonardo Aguiar Tavares in Portugal.',
        'The application allows users to control digital features and commands linked to the ARMIAC robotic hand.',
        'The user must use the application responsibly, safely, and in accordance with the law.',
        'The ARMIAC name, logo, design, code, and visual identity belong to the project owner.',
        'ARMiAC respects user privacy and aims to process personal data securely, in line with GDPR when applicable.',
        'Use robotic hand commands carefully. The user is responsible for testing the system in a safe environment.',
        'Contact: kevinleonardomail@gmail.com | Website: https://kevinleonardohub.github.io/portfolio',
    ]

```

```
];
```

```
String get speechServerDisconnectedError => isPt  
  ? 'O motor de voz desligou neste telemóvel. A app vai tentar sem forçar o modo local.  
  Toque novamente em Controlo por voz.'  
  : 'The voice engine disconnected on this phone. The app will try without forcing local  
  mode. Tap Voice control again.';  
}
```

Diretório – proj_armiac/lib/home/control_page.dart

```
import 'package:flutter/material.dart';  
  
import '../app_language.dart';  
import '../l10n.dart';  
import '../services/esp32_wifi_service.dart';  
  
/// Página de controlo manual da mão robótica.  
class ControlPage extends StatefulWidget {  
  const ControlPage({super.key});  
  
  @override  
  State<ControlPage> createState() => _ControlPageState();  
}  
  
/// Mantém o estado visual dos dedos e envia comandos para o ESP32.  
class _ControlPageState extends State<ControlPage> {  
  bool handClosed = false;  
  bool peaceMode = false;  
  bool indexClosed = false;  
  bool middleClosed = false;  
  bool ringPinkyClosed = false;  
  bool thumbClosed = false;  
  String lastGestureKey = 'open_hand';  
  String forearmKey = 'forearm_neutral';  
  bool _demoRunning = false;  
  
  L10n get _l10n => L10n(appSettings.language);  
  
  /// Converte a chave interna de um gesto no texto apresentado ao utilizador.  
  String _gestureLabel(String key) {  
    final l10n = _l10n;  
    switch (key) {  
      case 'open_hand':  
        return l10n.openHandState;  
      case 'close_hand':
```

```

        return l10n.closedFistState;
    case 'peace':
        return l10n.peaceSign;
    case 'two':
        return l10n.numberTwo;
    case 'point':
        return l10n.point;
    case 'three':
        return l10n.numberThree;
    case 'four':
        return l10n.numberFour;
    case 'thumbs_up':
        return l10n.thumbsUp;
    case 'ok':
        return l10n.okSign;
    case 'wave':
        return l10n.wave;
    case 'demo':
        return l10n.demoState;
    case 'index_closed':
        return l10n.indexClosed;
    case 'index_open':
        return l10n.indexOpen;
    case 'middle_closed':
        return l10n.middleClosed;
    case 'middle_open':
        return l10n.middleOpen;
    case 'ring_pinky_closed':
        return l10n.ringPinkyClosed;
    case 'ring_pinky_open':
        return l10n.ringPinkyOpen;
    case 'thumb_closed':
        return l10n.thumbClosed;
    case 'thumb_open':
        return l10n.thumbOpen;
    }
    return key;
}

```

/// Converte a posição do antebraço num texto legível.

```

String_forearmLabel(String key) {
    final l10n = _l10n;
    switch (key) {
        case 'palm_up':
            return l10n.palmUpState;
    case 'palm_down':
        return l10n.palmDownState;
    }
}

```

```

    default:
        return l10n.forearmCenterState;
    }
}

/// Envia uma ação para o ESP32 e devolve se o pedido correu bem.
Future<bool> _send(String action) async {
    if (!Esp32WifiService.instance.isConnected.value) {
        ScaffoldMessenger.of(context).showSnackBar(SnackBar(content:
Text(_l10n.connectFirst)));
        return false;
    }

    try {
        await Esp32WifiService.instance.action(action);
        return true;
    } catch (e) {
        if (!mounted) return false;
        ScaffoldMessenger.of(context).showSnackBar(SnackBar(content:
Text(_l10n.actionFailed)));
        return false;
    }
}

/// Atualiza o estado local para representar a mão aberta.
void _setOpenHand() {
    handClosed = false;
    peaceMode = false;
    indexClosed = false;
    middleClosed = false;
    ringPinkyClosed = false;
    thumbClosed = false;
    lastGestureKey = 'open_hand';
}

/// Atualiza o estado local para representar a mão fechada.
void _setClosedHand() {
    handClosed = true;
    peaceMode = false;
    indexClosed = true;
    middleClosed = true;
    ringPinkyClosed = true;
    thumbClosed = true;
    lastGestureKey = 'close_hand';
}

/// Sincroniza a interface com o gesto que acabou de ser executado.

```

```

void _applyActionState(String action) {
    switch (action) {
        case 'open_hand':
            _setOpenHand();
            break;
        case 'close_hand':
            _setClosedHand();
            break;
        case 'peace':
            handClosed = false;
            peaceMode = true;
            indexClosed = false;
            middleClosed = false;
            ringPinkyClosed = true;
            thumbClosed = true;
            lastGestureKey = 'peace';
            break;
        case 'two':
            handClosed = false;
            peaceMode = false;
            indexClosed = false;
            middleClosed = false;
            ringPinkyClosed = true;
            thumbClosed = true;
            lastGestureKey = 'two';
            break;
        case 'point':
            handClosed = false;
            peaceMode = false;
            indexClosed = false;
            middleClosed = true;
            ringPinkyClosed = true;
            thumbClosed = true;
            lastGestureKey = 'point';
            break;
        case 'three':
            handClosed = false;
            peaceMode = false;
            indexClosed = false;
            middleClosed = false;
            ringPinkyClosed = true;
            thumbClosed = false;
            lastGestureKey = 'three';
            break;
        case 'four':
            handClosed = false;
            peaceMode = false;
    }
}

```

```

    indexClosed = false;
    middleClosed = false;
    ringPinkyClosed = false;
    thumbClosed = true;
    lastGestureKey = 'four';
    break;
case 'thumbs_up':
    handClosed = false;
    peaceMode = false;
    indexClosed = true;
    middleClosed = true;
    ringPinkyClosed = true;
    thumbClosed = false;
    lastGestureKey = 'thumbs_up';
    break;
case 'ok':
    handClosed = false;
    peaceMode = false;
    indexClosed = true;
    middleClosed = false;
    ringPinkyClosed = false;
    thumbClosed = false;
    lastGestureKey = 'ok';
    break;
case 'wave':
    handClosed = false;
    peaceMode = false;
    indexClosed = false;
    middleClosed = false;
    ringPinkyClosed = false;
    thumbClosed = false;
    lastGestureKey = 'wave';
    break;
case 'palm_up':
case 'palm_down':
case 'forearm_neutral':
    forearmKey = action;
    break;
case 'demo':
    lastGestureKey = 'demo';
    break;
}
}

```

```

/// Envia o comando e só atualiza o ecrã se o ESP32 responder corretamente.
Future<bool> _sendActionAndUpdate(String action) async {
    final ok = await _send(action);

```

```

if (!ok || !mounted) return false;
setState(() => _applyActionState(action));
return true;
}

```

/// Executa um gesto simples, bloqueando ações durante a demonstração.

```

Future<void> _gesture(String action) async {
  if (!_demoRunning) return;

```

```

  if (action == 'demo') {
    await _runDemo();
    return;
  }

```

```

  if (action == 'wave') {
    final opened = await _sendActionAndUpdate('open_hand');
    if (!opened) return;
    await Future<void>.delayed(const Duration(milliseconds: 180));
    await _sendActionAndUpdate('wave');
    return;
  }

```

```

  final sent = await _sendActionAndUpdate(action);
  if (!sent || !mounted) return;

```

```

  final isHandGesture = ![
    'palm_up',
    'palm_down',
    'forearm_neutral',
  ].contains(action);

```

```

  if (isHandGesture && forearmKey != 'forearm_neutral') {
    await Future<void>.delayed(const Duration(milliseconds: 120));
    await _send(forearmKey);
  }
}

```

/// Corre a sequência de demonstração e mantém o estado final coerente.

```

Future<void> _runDemo() async {
  if (!_demoRunning) return;
  setState(() => _demoRunning = true);

```

```

  final sequence = [
    'open_hand',
    'palm_up',
    'peace',
    'two',

```

```

    'palm_down',
    'point',
    'forearm_neutral',
    'three',
    'four',
    'palm_up',
    'thumbs_up',
    'palm_down',
    'ok',
    'forearm_neutral',

    'open_hand',
    'wave',
    'forearm_neutral',
    'open_hand',
];

for (final action in sequence) {
  if (!mounted) return;
  final ok = await _sendActionAndUpdate(action);
  if (!ok) break;
  await Future<void>.delayed(const Duration(milliseconds: 450));
}

if (mounted) {
  setState(() {
    _demoRunning = false;
    lastGestureKey = 'demo';
  });
}
}

```

/// Alterna entre mão aberta e mão fechada.

```

Future<void> _toggleHand() async {
  await _gesture(handClosed ? 'open_hand' : 'close_hand');
}

```

/// Alterna o estado do indicador.

```

Future<void> _toggleIndex() async {
  final ok = await _send(indexClosed ? 'open_index' : 'close_index');
  if (!ok || !mounted) return;
  setState(() {
    indexClosed = !indexClosed;
    handClosed = false;
    peaceMode = false;
  });
  lastGestureKey = indexClosed ? 'index_closed' : 'index_open';
}

```



```

if (forearmKey != 'forearm_neutral') {
  await Future<void>.delayed(const Duration(milliseconds: 120));
  await _send(forearmKey);
}
}

```

/// Alterna o estado do dedo médio.

```

Future<void> _toggleMiddle() async {
  final ok = await _send(middleClosed ? 'open_middle' : 'close_middle');
  if (!ok || !mounted) return;
  setState(() {
    middleClosed = !middleClosed;
    handClosed = false;
    peaceMode = false;
    lastGestureKey = middleClosed ? 'middle_closed' : 'middle_open';
  });
}

```

```

if (forearmKey != 'forearm_neutral') {
  await Future<void>.delayed(const Duration(milliseconds: 120));
  await _send(forearmKey);
}
}

```

/// Alterna o estado do anelar e do mindinho.

```

Future<void> _toggleRingPinky() async {
  final ok = await _send(ringPinkyClosed ? 'open_ringpinky' : 'close_ringpinky');
  if (!ok || !mounted) return;
  setState(() {
    ringPinkyClosed = !ringPinkyClosed;
    handClosed = false;
    peaceMode = false;
    lastGestureKey = ringPinkyClosed ? 'ring_pinky_closed' : 'ring_pinky_open';
  });
}

```

```

if (forearmKey != 'forearm_neutral') {
  await Future<void>.delayed(const Duration(milliseconds: 120));
  await _send(forearmKey);
}
}

```

/// Alterna o estado do polegar.

```

Future<void> _toggleThumb() async {
  final ok = await _send(thumbClosed ? 'open_thumb' : 'close_thumb');
  if (!ok || !mounted) return;
  setState(() {
    thumbClosed = !thumbClosed;
  });
}

```

```

    handClosed = false;
    peaceMode = false;
    lastGestureKey = thumbClosed ? 'thumb_closed' : 'thumb_open';
  });

```

```

  if (forearmKey != 'forearm_neutral') {
    await Future<void>.delayed(const Duration(milliseconds: 120));
    await _send(forearmKey);
  }
}

```

/// Cria um botão reutilizável para comandos rápidos.

```

Widget _actionButton({required String label, required IconData icon, required
VoidCallback? onPressed, Color? color}) {
  return SizedBox(
    width: double.infinity,
    child: ElevatedButton.icon(
      onPressed: onPressed,
      icon: Icon(icon),
      style: ElevatedButton.styleFrom(backgroundColor: color),
      label: Text(label),
    ),
  );
}

```

/// Cria um botão que mostra se uma parte da mão está aberta ou fechada.

```

Widget _toggleButton({required String label, required bool state, required VoidCallback?
onPressed}) {
  return SizedBox(
    width: double.infinity,
    child: ElevatedButton(
      onPressed: onPressed,
      style: ElevatedButton.styleFrom(backgroundColor: state ? Colors.green : null),
      child: Text(state ? '$label ': label),
    ),
  );
}

```

@override

```

Widget build(BuildContext context) {
  return AnimatedBuilder(
    animation: appSettings,
    builder: (context, _) {
      final l10n = _l10n;
      return Scaffold(
        appBar: AppBar(title: Text(l10n.handControl)),
        body: ValueListenableBuilder<bool>(

```

```

valueListenable: Esp32WifiService.instance.isConnected,
builder: (context, connected, _) => SingleChildScrollView(
  padding: const EdgeInsets.all(16),
  child: Column(
    children: [
      Container(
        width: double.infinity,
        padding: const EdgeInsets.all(12),
        decoration: BoxDecoration(
          border: Border.all(color: connected ? Colors.green : Colors.red),
          borderRadius: BorderRadius.circular(12),
        ),
        child: Row(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Icon(connected ? Icons.wifi : Icons.wifi_off),
            const SizedBox(width: 8),
            Text(
              connected ? I10n.connectedToArmiac : I10n.notConnectedToArmiac,
              style: const TextStyle(fontWeight: FontWeight.bold),
            ),
          ],
        ),
      ),
      const SizedBox(height: 12),
      Text('${I10n.status}: $_gestureLabel(lastGestureKey)', style: const
TextStyle(fontWeight: FontWeight.bold)),
      const SizedBox(height: 4),
      Text('${I10n.forearm}: $_forearmLabel(forearmKey)', style: const
TextStyle(fontWeight: FontWeight.bold)),
      const SizedBox(height: 14),
      _actionButton(label: I10n.openCloseHand, icon: Icons.pan_tool_alt, onPressed:
connected && !_demoRunning ? _toggleHand : null),
      const SizedBox(height: 10),
      _actionButton(label: I10n.peaceSign, icon: Icons.gesture, onPressed:
connected && !_demoRunning ? () => _gesture('peace') : null),
      const SizedBox(height: 10),
      _actionButton(label: I10n.numberTwo, icon: Icons.looks_two, onPressed:
connected && !_demoRunning ? () => _gesture('two') : null),
      const SizedBox(height: 10),
      _actionButton(label: I10n.point, icon: Icons.touch_app, onPressed: connected
&& !_demoRunning ? () => _gesture('point') : null),
      const SizedBox(height: 10),
      _actionButton(label: I10n.numberThree, icon: Icons.looks_3, onPressed:
connected && !_demoRunning ? () => _gesture('three') : null),
      const SizedBox(height: 10),

```

```

        _actionButton(label: l10n.numberFour, icon: Icons.looks_4, onPressed:
connected && !_demoRunning ? () => _gesture('four') : null),
        const SizedBox(height: 10),
        _actionButton(label: l10n.thumbsUp, icon: Icons.thumb_up, onPressed:
connected && !_demoRunning ? () => _gesture('thumbs_up') : null),
        const SizedBox(height: 10),
        _actionButton(label: l10n.okSign, icon: Icons.check_circle_outline, onPressed:
connected && !_demoRunning ? () => _gesture('ok') : null),
        const SizedBox(height: 10),
        _actionButton(label: l10n.wave, icon: Icons.waving_hand, onPressed:
connected && !_demoRunning ? () => _gesture('wave') : null),
        const SizedBox(height: 10),
        _actionButton(label: l10n.palmUp, icon: Icons.keyboard_arrow_up, onPressed:
connected && !_demoRunning ? () => _gesture('palm_up') : null),
        const SizedBox(height: 10),
        _actionButton(label: l10n.palmDown, icon: Icons.keyboard_arrow_down,
onPressed: connected && !_demoRunning ? () => _gesture('palm_down') : null),
        const SizedBox(height: 10),
        _actionButton(label: l10n.forearmNeutral, icon: Icons.swap_vert, onPressed:
connected && !_demoRunning ? () => _gesture('forearm_neutral') : null),
        const SizedBox(height: 10),
        _actionButton(label: l10n.automaticDemo, icon: Icons.play_arrow, onPressed:
connected && !_demoRunning ? () => _gesture('demo') : null),
        const SizedBox(height: 20),
        _toggleButton(label: l10n.indexFinger, state: indexClosed, onPressed:
connected && !_demoRunning ? _toggleIndex : null),
        const SizedBox(height: 10),
        _toggleButton(label: l10n.middleFinger, state: middleClosed, onPressed:
connected && !_demoRunning ? _toggleMiddle : null),
        const SizedBox(height: 10),
        _toggleButton(label: l10n.ringPinky, state: ringPinkyClosed, onPressed:
connected && !_demoRunning ? _toggleRingPinky : null),
        const SizedBox(height: 10),
        _toggleButton(label: l10n.thumb, state: thumbClosed, onPressed: connected
&& !_demoRunning ? _toggleThumb : null),
    ],
  ),
),
),
),
);
},
);
}
}

```

Diretório – proj_armiac/lib/home/home_page.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_tts/flutter_tts.dart';
import 'package:speech_to_text/speech_to_text.dart' as stt;
import 'package:shared_preferences/shared_preferences.dart';
```

```
import '../app_language.dart';
import '../l10n.dart';
import '../services/esp32_wifi_service.dart';
import '../services/local_command_ai_service.dart';
import 'control_page.dart';
import 'tutorial_dialog.dart';
```

/// Página inicial, com ligação ao ARMIAC, voz, IA local e atalhos principais.

```
class HomePage extends StatefulWidget {
  const HomePage({super.key});

  @override
  State<HomePage> createState() => _HomePageState();
}
```

/// Comandos internos que a aplicação consegue transformar em ações para o ESP32.

```
enum _ArmiacCommand {
  openHand,
  closeHand,
  peace,
  two,
  point,
  three,
  four,
  thumbsUp,
  ok,
  wave,
  palmUp,
  palmDown,
  forearmNeutral,
  demo,
  openIndex,
  closeIndex,
  openMiddle,
  closeMiddle,
  openRingPinky,
  closeRingPinky,
  openThumb,
  closeThumb,
}
```

```

class _HomePageState extends State<HomePage> {
  bool _loading = false;
  bool _listening = false;
  bool _demoRunning = false;
  String _lastVoiceText = "";
  String _aiFeedback = "";

  stt.SpeechToText _speech = stt.SpeechToText();
  final FlutterTts _tts = FlutterTts();

  bool handClosed = false;
  bool peaceMode = false;
  bool indexClosed = false;
  bool middleClosed = false;
  bool ringPinkyClosed = false;
  bool thumbClosed = false;
  String forearmKey = 'forearm_neutral';

  @override
  /// Prepara a voz e mostra o tutorial inicial quando necessário.
  void initState() {
    super.initState();
    _configureTts();
    WidgetsBinding.instance.addPostFrameCallback((_) {
      _openTutorialOnce();
    });
  }

  @override
  /// Liberta recursos de voz antes de sair da página.
  void dispose() {
    _speech.stop();
    _tts.stop();
    super.dispose();
  }

  L10n get _l10n => L10n(appSettings.language);

  /// Configura a voz que responde ao utilizador.
  Future<void> _configureTts() async {
    await _tts.setLanguage(appSettings.language.ttsLocale);
    await _tts.setSpeechRate(0.9);
    await _tts.setPitch(1.0);
    await _tts.awaitSpeakCompletion(true);
  }
}

```

```

/// Fala uma resposta curta sem deixar falas antigas acumuladas.
Future<void> _speakFast(String text) async {
  await _tts.stop();
  await _configureTts();
  await _tts.speak(text);
}

/// Mostra o tutorial apenas na primeira utilização da aplicação.
Future<void> _openTutorialOnce() async {
  final prefs = await SharedPreferences.getInstance();
  final alreadyShown = prefs.getBool('tutorial_shown_v1_1_4') ?? false;

  if (!mounted || alreadyShown) return;

  await prefs.setBool('tutorial_shown_v1_1_4', true);
  _openTutorial();
}

/// Abre manualmente o tutorial.
void _openTutorial() {
  showDialog<void>(
    context: context,
    builder: (_) => const TutorialDialog(),
  );
}

/// Testa a comunicação com o ESP32 e atualiza o estado de ligação.
Future<void> _connect() async {
  setState(() => _loading = true);
  try {
    await Esp32WifiService.instance.connect();
    if (!mounted) return;
    ScaffoldMessenger.of(
      context,
    ).showSnackBar(SnackBar(content: Text(_l10n.connectedSnack)));
  } catch (e) {
    if (!mounted) return;
    ScaffoldMessenger.of(
      context,
    ).showSnackBar(SnackBar(content: Text(_l10n.connectFirst)));
  } finally {
    if (mounted) setState(() => _loading = false);
  }
}

/// Envia um comando HTTP para o ESP32.
Future<bool> _send(String action) async {

```

```

if (!Esp32WifiService.instance.isConnected.value) {
    ScaffoldMessenger.of(
        context,
    ).showSnackBar(SnackBar(content: Text(_l10n.connectFirst)));
    return false;
}

try {
    await Esp32WifiService.instance.action(action);
    return true;
} catch (e) {
    if (!mounted) return false;
    ScaffoldMessenger.of(
        context,
    ).showSnackBar(SnackBar(content: Text(_l10n.actionFailed)));
    return false;
}
}

```

/// Marca todos os dedos como abertos na interface.

```

void _setOpenHand() {
    handClosed = false;
    peaceMode = false;
    indexClosed = false;
    middleClosed = false;
    ringPinkyClosed = false;
    thumbClosed = false;
}

```

/// Marca todos os dedos como fechados na interface.

```

void _setClosedHand() {
    handClosed = true;
    peaceMode = false;
    indexClosed = true;
    middleClosed = true;
    ringPinkyClosed = true;
    thumbClosed = true;
}

```

/// Atualiza os estados locais de acordo com o gesto executado.

```

void _applyGestureState(String action) {
    switch (action) {
        case 'open_hand':
            _setOpenHand();
            break;
        case 'close_hand':
            _setClosedHand();
    }
}

```



```
break;
case 'peace':
    handClosed = false;
    peaceMode = true;
    indexClosed = false;
    middleClosed = false;
    ringPinkyClosed = true;
    thumbClosed = true;
    break;
case 'two':
    handClosed = false;
    peaceMode = false;
    indexClosed = false;
    middleClosed = false;
    ringPinkyClosed = true;
    thumbClosed = true;
    break;
case 'point':
    handClosed = false;
    peaceMode = false;
    indexClosed = false;
    middleClosed = true;
    ringPinkyClosed = true;
    thumbClosed = true;
    break;
case 'three':
    handClosed = false;
    peaceMode = false;
    indexClosed = false;
    middleClosed = false;
    ringPinkyClosed = true;
    thumbClosed = false;
    break;
case 'four':
    handClosed = false;
    peaceMode = false;
    indexClosed = false;
    middleClosed = false;
    ringPinkyClosed = false;
    thumbClosed = true;
    break;
case 'thumbs_up':
    handClosed = false;
    peaceMode = false;
    indexClosed = true;
    middleClosed = true;
    ringPinkyClosed = true;
```

```

thumbClosed = false;
    break;
case 'ok':
    handClosed = false;
    peaceMode = false;
    indexClosed = true;
    middleClosed = false;
    ringPinkyClosed = false;
    thumbClosed = false;
    break;
case 'wave':
    _setOpenHand();
    break;
case 'palm_up':
case 'palm_down':
case 'forearm_neutral':
    forearmKey = action;
    break;
case 'demo':
    break;
}
}

```

/// Envia o gesto e atualiza a interface apenas quando o envio funciona.

```

Future<bool> _sendGestureAndUpdate(String action) async {
    final ok = await _send(action);
    if (!ok || !mounted) return false;
    setState(() => _applyGestureState(action));
    return true;
}

```

/// Executa um gesto completo, evitando conflitos com a demonstração.

```

Future<void> _runGesture(String action) async {
    if (!_demoRunning) return;

    if (action == 'demo') {
        await _runDemo();
        return;
    }

    if (action == 'wave') {
        final opened = await _sendGestureAndUpdate('open_hand');
        if (!opened) return;
        await Future<void>.delayed(const Duration(milliseconds: 180));
        await _sendGestureAndUpdate('wave');
        return;
    }
}

```

```

final sent = await _sendGestureAndUpdate(action);
if (!sent || !mounted) return;

final isHandGesture = ![
  'palm_up',
  'palm_down',
  'forearm_neutral',
].contains(action);

if (isHandGesture && forearmKey != 'forearm_neutral') {
  await Future<void>.delayed(const Duration(milliseconds: 120));
  await _send(forearmKey);
}
}

```

/// Executa a sequência automática de demonstração.

```

Future<void> _runDemo() async {
  if (_demoRunning) return;
  setState(() => _demoRunning = true);

```

```

final sequence = [
  'open_hand',
  'palm_up',
  'peace',
  'two',
  'palm_down',
  'point',
  'forearm_neutral',
  'three',
  'four',
  'palm_up',
  'thumbs_up',
  'palm_down',
  'ok',
  'forearm_neutral',

  'open_hand',
  'wave',
  'forearm_neutral',
  'open_hand',
];

```

```

for (final action in sequence) {
  if (!mounted) return;
  final ok = await _sendGestureAndUpdate(action);
  if (!ok) break;

```

```

await Future<void>.delayed(const Duration(milliseconds: 450));
}

if (mounted) {
  setState(() => _demoRunning = false);
}
}

/// Executa ações individuais dos dedos e atualiza só a parte afetada.
Future<void> _runFingerAction(String action) async {
  final ok = await _send(action);
  if (!ok || !mounted) return;

  setState(() {
    handClosed = false;
    peaceMode = false;

    switch (action) {
      case 'open_index':
        indexClosed = false;
        break;
      case 'close_index':
        indexClosed = true;
        break;
      case 'open_middle':
        middleClosed = false;
        break;
      case 'close_middle':
        middleClosed = true;
        break;
      case 'open_ringpinky':
        ringPinkyClosed = false;
        break;
      case 'close_ringpinky':
        ringPinkyClosed = true;
        break;
      case 'open_thumb':
        thumbClosed = false;
        break;
      case 'close_thumb':
        thumbClosed = true;
        break;
    }
  });

  if (forearmKey != 'forearm_neutral') {
    await Future<void>.delayed(const Duration(milliseconds: 120));
  }
}

```

```
await _send(forearmKey);
}
}
```

/// Alterna rapidamente entre abrir e fechar a mão.

```
Future<void> _toggleHand() async {
  await _runGesture(handClosed ? 'open_hand' : 'close_hand');
}
```

/// Liga o comando interpretado à ação real que deve ser enviada ao ESP32.

```
Future<void> _executeCommand(_ArmiacCommand command) async {
  switch (command) {
    case _ArmiacCommand.openHand:
      await _runGesture('open_hand');
      break;
    case _ArmiacCommand.closeHand:
      await _runGesture('close_hand');
      break;
    case _ArmiacCommand.peace:
      await _runGesture('peace');
      break;
    case _ArmiacCommand.two:
      await _runGesture('two');
      break;
    case _ArmiacCommand.point:
      await _runGesture('point');
      break;
    case _ArmiacCommand.three:
      await _runGesture('three');
      break;
    case _ArmiacCommand.four:
      await _runGesture('four');
      break;
    case _ArmiacCommand.thumbsUp:
      await _runGesture('thumbs_up');
      break;
    case _ArmiacCommand.ok:
      await _runGesture('ok');
      break;
    case _ArmiacCommand.wave:
      await _runGesture('wave');
      break;
    case _ArmiacCommand.palmUp:
      await _runGesture('palm_up');
      break;
    case _ArmiacCommand.palmDown:
      await _runGesture('palm_down');
```

```

break;
case _ArmiacCommand.forearmNeutral:
    await _runGesture('forearm_neutral');
    break;
case _ArmiacCommand.demo:
    await _runGesture('demo');
    break;
case _ArmiacCommand.openIndex:
    await _runFingerAction('open_index');
    break;
case _ArmiacCommand.closeIndex:
    await _runFingerAction('close_index');
    break;
case _ArmiacCommand.openMiddle:
    await _runFingerAction('open_middle');
    break;
case _ArmiacCommand.closeMiddle:
    await _runFingerAction('close_middle');
    break;
case _ArmiacCommand.openRingPinky:
    await _runFingerAction('open_ringpinky');
    break;
case _ArmiacCommand.closeRingPinky:
    await _runFingerAction('close_ringpinky');
    break;
case _ArmiacCommand.openThumb:
    await _runFingerAction('open_thumb');
    break;
case _ArmiacCommand.closeThumb:
    await _runFingerAction('close_thumb');
    break;
}
}

_ArmiacCommand? _commandFromAction(String action) {
switch (action) {
case 'open_hand':
    return _ArmiacCommand.openHand;
case 'close_hand':
    return _ArmiacCommand.closeHand;
case 'peace':
    return _ArmiacCommand.peace;
case 'two':
    return _ArmiacCommand.two;
case 'point':
    return _ArmiacCommand.point;
case 'three':

```

```

return _ArmiacCommand.three;
    case 'four':
        return _ArmiacCommand.four;
    case 'thumbs_up':
        return _ArmiacCommand.thumbsUp;
    case 'ok':
        return _ArmiacCommand.ok;
    case 'wave':
        return _ArmiacCommand.wave;
    case 'palm_up':
        return _ArmiacCommand.palmUp;
    case 'palm_down':
        return _ArmiacCommand.palmDown;
    case 'forearm_neutral':
        return _ArmiacCommand.forearmNeutral;
    case 'demo':
        return _ArmiacCommand.demo;
    case 'open_index':
        return _ArmiacCommand.openIndex;
    case 'close_index':
        return _ArmiacCommand.closeIndex;
    case 'open_middle':
        return _ArmiacCommand.openMiddle;
    case 'close_middle':
        return _ArmiacCommand.closeMiddle;
    case 'open_ringpinky':
        return _ArmiacCommand.openRingPinky;
    case 'close_ringpinky':
        return _ArmiacCommand.closeRingPinky;
    case 'open_thumb':
        return _ArmiacCommand.openThumb;
    case 'close_thumb':
        return _ArmiacCommand.closeThumb;
    }
    return null;
}

/// Escolhe a frase que será dita após cada comando reconhecido.
String _commandToSpeech(_ArmiacCommand command) {
    switch (command) {
        case _ArmiacCommand.openHand:
            return _l10n.speechFor('open_hand');
        case _ArmiacCommand.closeHand:
            return _l10n.speechFor('close_hand');
        case _ArmiacCommand.peace:
            return _l10n.speechFor('peace');
        case _ArmiacCommand.two:

```

```

return _l10n.speechFor('two');
    case _ArmiacCommand.point:
        return _l10n.speechFor('point');
    case _ArmiacCommand.three:
        return _l10n.speechFor('three');
    case _ArmiacCommand.four:
        return _l10n.speechFor('four');
    case _ArmiacCommand.thumbsUp:
        return _l10n.speechFor('thumbs_up');
    case _ArmiacCommand.ok:
        return _l10n.speechFor('ok');
    case _ArmiacCommand.wave:
        return _l10n.speechFor('wave');
    case _ArmiacCommand.palmUp:
        return _l10n.speechFor('palm_up');
    case _ArmiacCommand.palmDown:
        return _l10n.speechFor('palm_down');
    case _ArmiacCommand.forearmNeutral:
        return _l10n.speechFor('forearm_neutral');
    case _ArmiacCommand.demo:
        return _l10n.speechFor('demo');
    case _ArmiacCommand.openIndex:
        return _l10n.speechFor('open_index');
    case _ArmiacCommand.closeIndex:
        return _l10n.speechFor('close_index');
    case _ArmiacCommand.openMiddle:
        return _l10n.speechFor('open_middle');
    case _ArmiacCommand.closeMiddle:
        return _l10n.speechFor('close_middle');
    case _ArmiacCommand.openRingPinky:
        return _l10n.speechFor('open_ringpinky');
    case _ArmiacCommand.closeRingPinky:
        return _l10n.speechFor('close_ringpinky');
    case _ArmiacCommand.openThumb:
        return _l10n.speechFor('open_thumb');
    case _ArmiacCommand.closeThumb:
        return _l10n.speechFor('close_thumb');
}
}

/// Interpreta o texto do utilizador e executa o comando correspondente.
Future<void> _runAiCommand(String text) async {
    setState(() {
        _aiFeedback = 'Texto ouvido: "$text"\nA interpretar o comando...';
    });

    final action = LocalCommandAiService.classify(text);

```



```

if (action == 'unknown') {
  setState(() {
    _aiFeedback = 'Não consegui entender: "$text"';
  });

  await _speakFast(_l10n.didNotUnderstand);
  return;
}

final command = _commandFromAction(action);

if (command == null) {
  setState(() {
    _aiFeedback = 'Comando inválido: $action';
  });
  return;
}

final spoken = _commandToSpeech(command);

setState(() {
  _aiFeedback =
    'Texto ouvido: "$text"\nComando identificado: $action\nResposta: $spoken';
});

await _speakFast(spoken);
await _executeCommand(command);
}

/// Escolhe o melhor idioma disponível para o reconhecimento de voz.
Future<String> _bestSpeechLocale() async {
  final preferred = appSettings.language == AppLanguage.portuguese
    ? 'pt_BR'
    : 'en_US';
  try {
    final locales = await _speech.locales();
    final exact = locales.where(
      (locale) => locale.localeId.toLowerCase() == preferred.toLowerCase(),
    );
    if (exact.isNotEmpty) return exact.first.localeId;

    final prefix = appSettings.language == AppLanguage.portuguese
      ? 'pt'
      : 'en';
    final sameLanguage = locales.where(
      (locale) => locale.localeId.toLowerCase().startsWith(prefix),
    );
  }
}

```

```

);
    if (sameLanguage.isNotEmpty) return sameLanguage.first.localeId;
  } catch (_) {}

}
return preferred;
}

/// Liga o microfone, recolhe a frase e envia o texto para interpretação.
Future<void> _listenVoiceCommand() async {
  if (_listening) {
    await _speech.stop();
    setState(() => _listening = false);
    return;
  }

  try {
    await _speech.cancel();
  } catch (_) {}
  _speech = stt.SpeechToText();

  final available = await _speech.initialize(
    onStatus: (status) {
      if ((status == 'done' || status == 'notListening') && mounted) {
        setState(() => _listening = false);
      }
    },
    onError: (error) {
      if (mounted) {
        setState(() {
          _listening = false;
          _aiFeedback = _speechErrorMessage(error.errorMsg);
        });
      }
    },
  );

  if (!available) {
    setState(() => _aiFeedback = _l10n.micUnavailable);
    return;
  }

  final localeId = await _bestSpeechLocale();

  setState(() {
    _listening = true;
    _lastVoiceText = "";
  });
}

```

```

_aiFeedback = '${_l10n.listening} ($localeId)';
});

try {
  await _speech.listen(
    localeId: localeId,
    onResult: (result) async {
      final words = result.recognizedWords;
      setState(() => _lastVoiceText = words);

      if (result.finalResult && words.trim().isEmpty) {
        await _speech.stop();
        if (mounted) setState(() => _listening = false);
        await _runAiCommand(words);
      }
    },
    listenFor: const Duration(seconds: 8),
    pauseFor: const Duration(seconds: 2),
    listenOptions: stt.SpeechListenOptions(
      onDevice: false,
      cancelOnError: true,
      partialResults: false,
    ),
  );
} catch (e) {
  if (mounted) {
    setState(() {
      _listening = false;
      _aiFeedback = _speechErrorMessage(e.toString());
    });
  }
}
}

```

/// Traduz erros do reconhecimento de voz para mensagens mais claras.

```

String _speechErrorMessage(String error) {
  final lower = error.toLowerCase();
  if (lower.contains('server') || lower.contains('disconnected')) {
    return _l10n.speechServerDisconnectedError;
  }
  if (lower.contains('network')) {
    return _l10n.speechNetworkError;
  }
  if (lower.contains('language') ||
    lower.contains('locale') ||
    lower.contains('not supported')) {
    return _l10n.speechLanguageError;
  }
}

```

```

}
return _l10n.micError(error);
}

/// Mostra uma lista de exemplos de comandos de voz.
void _openCommandHelp() {
  final l10n = _l10n;
  showModalBottomSheet<void>(
    context: context,
    isScrollControlled: true,
    builder: (context) => SafeArea(
      child: SingleChildScrollView(
        padding: const EdgeInsets.all(20),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.min,
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(
              l10n.commandsHelp,
              style: const TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
            ),
            const SizedBox(height: 8),
            Text(l10n.commandsHelpIntro),
            const SizedBox(height: 12),
            ...l10n.commandHelpItems.map(
              (item) => Padding(
                padding: const EdgeInsets.symmetric(vertical: 4),
                child: Text('• $item'),
              ),
            ),
          ],
        ),
      ),
    );
}

```

@override

```

Widget build(BuildContext context) {
  return AnimatedBuilder(
    animation: appSettings,
    builder: (context, _) {
      final l10n = _l10n;
      return ValueListenableBuilder<bool>(

```

```

valueListenable: Esp32WifiService.instance.isConnected,
builder: (context, connected, _) {
  return SingleChildScrollView(
    padding: const EdgeInsets.all(16),
    child: Column(
      mainAxisAlignment: MainAxisAlignment.center,
      children: [
        Text(
          connected ? I10n.connected : I10n.disconnected,
          style: const TextStyle(
            fontSize: 20,
            fontWeight: FontWeight.bold,
          ),
          textAlign: TextAlign.center,
        ),
        const SizedBox(height: 16),
        SizedBox(
          width: double.infinity,
          child: ElevatedButton.icon(
            onPressed: _loading ? null : _connect,
            icon: Icon(connected ? Icons.wifi : Icons.wifi_off),
            label: Text(
              _loading ? I10n.connecting : I10n.connectArmiac,
            ),
          ),
        ),
        const SizedBox(height: 12),
        SizedBox(
          width: double.infinity,
          child: ElevatedButton.icon(
            onPressed: connected ? _listenVoiceCommand : null,
            icon: Icon(_listening ? Icons.stop : Icons.mic),
            label: Text(
              _listening ? I10n.stopVoice : I10n.voiceControl,
            ),
          ),
        ),
        const SizedBox(height: 8),
        Wrap(
          alignment: WrapAlignment.center,
          spacing: 8,
          runSpacing: 4,
          children: [
            TextButton.icon(
              onPressed: _openTutorial,
              icon: const Icon(Icons.slideshow_outlined),
              label: Text(I10n.tutorial),
            ),
          ],
        ),
      ],
    ),
  ),
)

```

```

),
    TextButton.icon(
      onPressed: _openCommandHelp,
      icon: const Icon(Icons.help_outline),
      label: Text(l10n.help),
    ),
  ],
),
if (_lastVoiceText.isNotEmpty) ...[
  const SizedBox(height: 8),
  Text(
    '${l10n.voice}: $_lastVoiceText',
    textAlign: TextAlign.center,
  ),
],
if (_aiFeedback.isNotEmpty) ...[
  const SizedBox(height: 8),
  Text(_aiFeedback, textAlign: TextAlign.center),
],
const SizedBox(height: 20),
Center(
  child: Text(
    l10n.quickCommands,
    textAlign: TextAlign.center,
    style: const TextStyle(
      fontSize: 16,
      fontWeight: FontWeight.bold,
    ),
  ),
),
const SizedBox(height: 8),
Wrap(
  spacing: 8,
  runSpacing: 8,
  alignment: WrapAlignment.center,
  children: [
    ActionChip(
      label: Text(l10n.openCloseHand),
      onPressed: connected && !_demoRunning ? _toggleHand : null,
    ),
    ActionChip(
      label: Text(l10n.peace),
      onPressed: connected
        && !_demoRunning
        ? () => _runGesture('peace')
        : null,
    ),
  ],
),

```

```

ActionChip(
  label: Text(l10n.numberTwo),
  onPressed: connected
    && !_demoRunning
    ? () => _runGesture('two')
    : null,
),
ActionChip(
  label: Text(l10n.point),
  onPressed: connected
    && !_demoRunning
    ? () => _runGesture('point')
    : null,
),
ActionChip(
  label: Text(l10n.numberFour),
  onPressed: connected && !_demoRunning
    ? () => _runGesture('four')
    : null,
),
ActionChip(
  label: Text(l10n.thumbsUp),
  onPressed: connected && !_demoRunning
    ? () => _runGesture('thumbs_up')
    : null,
),
ActionChip(
  label: const Text('OK'),
  onPressed: connected && !_demoRunning ? () => _runGesture('ok') : null,
),
ActionChip(
  label: Text(l10n.wave),
  onPressed: connected && !_demoRunning ? () => _runGesture('wave') :
null,
),
ActionChip(
  label: Text(l10n.palmUpShort),
  onPressed: connected
    && !_demoRunning
    ? () => _runGesture('palm_up')
    : null,
),
ActionChip(
  label: Text(l10n.palmDownShort),
  onPressed: connected
    && !_demoRunning
    ? () => _runGesture('palm_down')

```

```

: null,
    ),
  ],
),
const SizedBox(height: 25),
SizedBox(
  width: double.infinity,
  child: ElevatedButton(
    onPressed: connected
      ? () {
        Navigator.push(
          context,
          MaterialPageRoute(
            builder: (_) => const ControlPage(),
          ),
        );
      }
    : null,
    child: Text(l10n.manualControl),
  ),
),
],
),
);
},
);
},
);
}
}

```

Diretório – proj_armiac/lib/home/tutorial_dialog.dart

```

import 'package:flutter/material.dart';

import '../app_language.dart';
import '../l10n.dart';

/// Janela com o tutorial rápido da aplicação.
class TutorialDialog extends StatefulWidget {
  const TutorialDialog({super.key});

  @override
  State<TutorialDialog> createState() => _TutorialDialogState();
}

```



```

class _TutorialDialogState extends State<TutorialDialog> {
  final PageController _controller = PageController();
  int _page = 0;

  L10n get _l10n => L10n(appSettings.language);

  @override
  /// Fecha corretamente o controlador das páginas.
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  /// Move o tutorial para a página pedida com animação.
  Future<void> _goTo(int page) async {
    await _controller.animateToPage(
      page,
      duration: const Duration(milliseconds: 250),
      curve: Curves.easeInOut,
    );
  }

  @override
  Widget build(BuildContext context) {
    final l10n = _l10n;
    final pages = [
      _TutorialCard(
        icon: Icons.power,
        title: l10n.tutorialStep1Title,
        body: l10n.tutorialStep1Body,
      ),
      _TutorialCard(
        icon: Icons.wifi,
        title: l10n.tutorialStep2Title,
        body: l10n.tutorialStep2Body,
        imageAsset: 'assets/tutorial/armiac_wifi.png',
      ),
    ];

    return Dialog(
      insetPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: 20),
      child: ConstrainedBox(
        constraints: BoxConstraints(
          maxWidth: 380,
          maxHeight: MediaQuery.of(context).size.height * 0.88,
        ),
      ),
    );
  }
}

```

```

child: Padding(
  padding: const EdgeInsets.fromLTRB(20, 18, 20, 18),
  child: Column(
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      Row(
        children: [
          Expanded(
            child: Text(
              l10n.tutorialTitle,
              style: const TextStyle(
                fontSize: 22,
                fontWeight: FontWeight.bold,
              ),
            ),
          IconButton(
            onPressed: () => Navigator.of(context).pop(),
            icon: const Icon(Icons.close),
          ),
        ],
      ),
      const SizedBox(height: 10),
      SizedBox(
        height: 335,
        child: PageView.builder(
          controller: _controller,
          itemCount: pages.length,
          onPageChanged: (value) => setState(() => _page = value),
          itemBuilder: (context, index) => pages[index],
        ),
      ),
      const SizedBox(height: 12),
      Row(
        mainAxisAlignment: MainAxisAlignment.center,
        children: List.generate(
          pages.length,
          (index) => AnimatedContainer(
            duration: const Duration(milliseconds: 200),
            margin: const EdgeInsets.symmetric(horizontal: 4),
            width: _page == index ? 20 : 8,
            height: 8,
            decoration: BoxDecoration(
              color: _page == index
                ? Theme.of(context).colorScheme.primary
                : Colors.grey.shade400,
              borderRadius: BorderRadius.circular(99),
            ),
          ),
        ),
      ),
    ],
  ),
),

```

```

),
    ),
    ),
    ),
    const SizedBox(height: 14),
    Row(
      children: [
        if (_page > 0)
          TextButton(
            onPressed: () => _goTo(_page - 1),
            child: Text(l10n.previous),
          )
        else
          const SizedBox.shrink(),
          const Spacer(),
          ElevatedButton(
            onPressed: _page == pages.length - 1
              ? () => Navigator.of(context).pop()
              : () => _goTo(_page + 1),
            child: Text(
              _page == pages.length - 1 ? l10n.finish : l10n.next,
            ),
          ),
        ),
      ],
    ),
  ],
),
),
),
),
),
);
}
}

```

/// Cartão visual usado em cada passo do tutorial.

```

class _TutorialCard extends StatelessWidget {
  const _TutorialCard({
    required this.icon,
    required this.title,
    required this.body,
    this.imageAsset,
  });

```

```

  final IconData icon;
  final String title;
  final String body;
  final String? imageAsset;

```

@override

```
Widget build(BuildContext context) {  
  return Container(  
    width: double.infinity,  
    padding: const EdgeInsets.all(16),  
    decoration: BoxDecoration(  
      color: Theme.of(context).colorScheme.surfaceVariant,  
      borderRadius: BorderRadius.circular(20),  
    ),  
    child: SingleChildScrollView(  
      physics: const BouncingScrollPhysics(),  
      child: Column(  
        mainAxisAlignment: MainAxisAlignment.center,  
        children: [  
          if (imageAsset != null) ...[  
            ClipRRect(  
              borderRadius: BorderRadius.circular(16),  
              child: Container(  
                width: double.infinity,  
                height: 175,  
                color: Colors.white,  
                child: Image.asset(  
                  imageAsset!,  
                  fit: BoxFit.contain,  
                ),  
              ),  
            ),  
          ],  
          const SizedBox(height: 16),  
          ] else ...[  
            CircleAvatar(  
              radius: 38,  
              child: Icon(icon, size: 38),  
            ),  
            const SizedBox(height: 22),  
          ],  
          Text(  
            title,  
            textAlign: TextAlign.center,  
            style: const TextStyle(fontSize: 20, fontWeight: FontWeight.bold),  
          ),  
          const SizedBox(height: 10),  
          Text(  
            body,  
            textAlign: TextAlign.center,  
            style: const TextStyle(fontSize: 15, height: 1.3),  
          ),  
        ],  
      ),  
    ),  
  ),  
);
```

```

),
),
);
}
}

```

Diretório – proj_armiac/lib/profile/profile_page.dart

```

import 'package:flutter/material.dart';

import '../app_language.dart';

/// Página simples reservada para dados de perfil.
class ProfilePage extends StatelessWidget {
  const ProfilePage({super.key});

  @override
  Widget build(BuildContext context) {
    final isPt = appSettings.language == AppLanguage.portuguese;
    return Center(
      child: Text(isPt ? 'Página Perfil' : 'Profile Page', style: const TextStyle(fontSize: 24)),
    );
  }
}

```

Diretório – proj_armiac/lib/services/ai_command_service.dart

```

import 'dart:convert';

import 'package:http/http.dart' as http;

import '../app_language.dart';

/// Serviço opcional para interpretar comandos com Gemini quando existir chave
/// configurada.
class AiCommandService {
  AiCommandService._();
  /// Instância única para evitar criar vários clientes do mesmo serviço.
  static final AiCommandService instance = AiCommandService._();
}

```

```

/// Chave lida em tempo de compilação através de --dart-define.
static const String geminiApiKey = String.fromEnvironment('GEMINI_API_KEY');

/// Lista de ações que a IA pode devolver e que o ESP32 sabe receber.
static const List<String> validCommands = [
  'open_hand',
  'close_hand',
  'peace',
  'point',
  'three',
  'thumbs_up',
  'ok',
  'wave',
  'palm_up',
  'palm_down',
  'forearm_neutral',
  'demo',
  'close_index',
  'open_index',
  'close_middle',
  'open_middle',
  'close_ringpinky',
  'open_ringpinky',
  'close_thumb',
  'open_thumb',
];

/// Indica se o serviço remoto pode ser usado.
bool get isConfigured => geminiApiKey.trim().isEmpty;

/// Envia a frase para o Gemini e tenta receber apenas uma ação válida.
Future<String?> understandWithGemini(String spokenText, AppLanguage language)
async {
  if (!isConfigured) return null;

  final uri = Uri.parse(
    'https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-
flash:generateContent?key=$geminiApiKey',
  );

  final languageName = language == AppLanguage.portuguese ? 'Portuguese' : 'English';
  final examples = language == AppLanguage.portuguese
    ? ""
    - abrir mão / abre a mão / resetar => open_hand

    - paz / vitória / número dois => peace

```

- apontar / número um => point
- número três / três dedos => three
- joinha / positivo => thumbs_up
- acenar / olá / adeus => wave
- palma para cima / supinação => palm_up
- palma para baixo / pronação => palm_down
- antebraço ao centro / posição neutra => forearm_neutral
- demonstração => demo

```

: ""

- open hand / reset / open all => open_hand
- close hand / fist / make a fist => close_hand
- peace / victory / number two => peace
- point / number one => point
- number three / three fingers => three
- thumbs up / like => thumbs_up
- wave / hello / bye => wave
- palm up / face up / supination => palm_up
- palm down / face down / pronation => palm_down
- neutral forearm / center forearm => forearm_neutral
- demo / demonstration => demo
",

```

final prompt = ""

You control a robotic hand called ARMIAC.

The app is currently set to \$languageName.

Only understand commands in \$languageName. If the user speaks another language, return UNKNOWN.

Return ONLY one command from this exact list:

`\${validCommands.join(', ')}`

Examples:

\$examples

If the phrase is unclear, return UNKNOWN.

User speech: "\$spokenText"

```

",

```

```

try {
  final response = await http
    .post(
      uri,
      headers: {'Content-Type': 'application/json'},
      body: jsonEncode({
        'contents': [
{
          'parts': [

```

```

        {'text': prompt}
    ]
  },
  ],
  'generationConfig': {
    'temperature': 0,
    'maxOutputTokens': 10,
  }
}),
)
.timeout(const Duration(seconds: 6));

if (response.statusCode != 200) return null;

final data = jsonDecode(response.body) as Map<String, dynamic>;
final text = data['candidates']?[0]?['content']?['parts']?[0]?['text']
    ?.toString()
    .trim()
    .toLowerCase();

if (text == null || text == 'unknown') return null;
return validCommands.contains(text) ? text : null;
} catch (_) {
  return null;
}
}
}
}

```

Diretório – proj_armiac/lib/services/esp32_wifi_service.dart

```

import 'dart:convert';

import 'package:flutter/foundation.dart';
import 'package:http/http.dart' as http;

/// Serviço responsável pela comunicação HTTP com o ESP32.
class Esp32WifiService {
  Esp32WifiService._();
  /// Instância única usada por todas as páginas.
  static final Esp32WifiService instance = Esp32WifiService._();

  /// Endereço padrão do ESP32 quando está a funcionar como ponto de acesso Wi-Fi.
  static const String baseUrl = 'http://192.168.4.1';
  final ValueNotifier<bool> isConnected = ValueNotifier<bool>(false);

  /// Verifica se o ESP32 responde e guarda o estado da ligação.
  Future<void> connect() async {
    final ok = await ping();
  }
}

```



```

    isConnected.value = ok;
    if (!ok) throw Exception('ARMIAC_WIFI_NOT_CONNECTED');
  }

  /// Faz um pedido simples para confirmar que o servidor está ativo.
  Future<bool> ping() async {
    try {
      final r = await http.get(Uri.parse('$baseUrl/ping')).timeout(const Duration(seconds: 2));
      return r.statusCode == 200;
    } catch (_) {
      return false;
    }
  }

  /// Lê o estado atual devolvido pelo ESP32.
  Future<Map<String, dynamic>?> status() async {
    try {
      final r = await http.get(Uri.parse('$baseUrl/status')).timeout(const Duration(seconds:
2));
      if (r.statusCode != 200) return null;
      return jsonDecode(r.body) as Map<String, dynamic>;
    } catch (_) {
      return null;
    }
  }

  /// Envia uma ação para o endpoint /action.
  Future<void> action(String a) async {
    final uri = Uri.parse('$baseUrl/action').replace(queryParameters: {'a': a});
    final r = await http.get(uri).timeout(const Duration(seconds: 3));
    if (r.statusCode != 200) {
      throw Exception('ARMIAC_ACTION_FAILED');
    }
  }
}

```

Diretório – proj_armiac/lib/services/local_command_ai_service.dart

```

import 'dart:math';

/// Interpretador local de comandos de voz, sem depender de internet.
class LocalCommandAiService {
  /// Frases muito específicas que identificam diretamente um comando.
  static const Map<String, List<String>> strongPhrases = {
    'open_hand': [
      'abrir mao',
      'abre mao',
      'abre a mao',

```

'abrir a mao',
'abrir os dedos',
'abre os dedos',
'abrir todos os dedos',
'abre todos os dedos',
'deixa a mao aberta',
'solta a mao',
'liberta a mao',
'abre completamente',
'abre totalmente a mao',
'estende a mao',
'deixa os dedos abertos',
'estica os dedos',
'mao aberta',
'open hand',
'open the hand',
'open fingers',
'open all fingers',
'release hand',
'stretch fingers',
'extend hand',
,
'close_hand': [
'fechar mao',
'fecha mao',
'fecha a mao',
'fechar a mao',
'feche a mao',
'feche os dedos',
'fecha os dedos',
'fechar os dedos',
'fechar todos os dedos',
'fecha todos os dedos',
'fazer punho',
'faz punho',
'fecha tudo',
'fecha os dedos todos',
'fecha a mao toda',
'junta os dedos',
'contraí a mao',
'aperta a mao',
'mao fechada',
'punho fechado',
'close hand',
'close the hand',
'make a fist',
'closed fist',

```
'squeeze hand',  
'close all fingers',  
'close the fingers',  
,  
'two': [  
  'numero dois',  
  'numero 2',  
  'dois',  
  'dois dedos',  
  'mostra dois',  
  'mostra dois dedos',  
  'faz dois',  
  'faz o dois',  
  'levanta dois dedos',  
  'number two',  
  'two',  
  'two fingers',  
  'show two',  
,  
'peace': [  
  'sinal de paz',  
  'faz paz',  
  'paz',  
  'vitoria',  
  'faz o v',  
  'sinal v',  
  'gesto v',  
  'faz vitoria',  
  'peace sign',  
  'victory sign',  
  'make a v',  
  'v sign',  
,  
'point': [  
  'apontar',  
  'aponta',  
  'aponta com o dedo',  
  'dedo indicador',  
  'deixa so o indicador',  
  'so o indicador',  
  'aponta em frente',  
  'aponta para frente',  
  'indicador aberto',  
  'numero um',  
'faz numero um',  
  'point',  
  'point finger',
```

```
'index finger',
'only index',
'point forward',
'number one',
],
'three': [
'tres dedos',
'numero tres',
'numero 3',
'faz tres',
'faz o tres',
'mostra tres dedos',
'levanta tres dedos',
'abre tres dedos',
'mostra o tres',
'three fingers',
'number three',
'show three',
],
'four': [
'quatro dedos',
'numero quatro',
'numero 4',
'quatro',
'faz quatro',
'faz o quatro',
'mostra quatro',
'mostra quatro dedos',
'levanta quatro dedos',
'four fingers',
'number four',
'show four',
],
'thumbs_up': [
'joinha',
'joia',
'faz joinha',
'faz positivo',
'sinal positivo',
'polegar para cima',
'dedao para cima',
'fixe',
'gosto',
'like',
'thumbs up',
'positive sign',
'thumb up',
```

```
],  
'ok': [  
  'ok',  
  'okay',  
  'faz ok',  
  'sinal de ok',  
  'esta bem',  
  'tudo bem',  
  'certo',  
  'perfeito',  
  'confirmar',  
  'confirma',  
  'gesto ok',  
  'okay sign',  
],  
'wave': [  
  'acenar',  
  'acena',  
  'dizer ola',  
  'diz ola',  
  'dizer adeus',  
  'diz adeus',  
  'cumprimentar',  
  'manda ola',  
  'diz xau',  
  'ola',  
  'adeus',  
  'cumprimenta',  
  'wave',  
  'say hello',  
  'say goodbye',  
  'greet',  
],  
'palm_up': [  
  'palma para cima',  
  'palma cima',  
  'mao para cima',  
  'vira a palma para cima',  
  'vira para cima',  
  'roda para cima',  
  'supinacao',  
  'palm up',  
  'rotate up',  
  'supination',  
],  
'palm_down': [  
  'palma para baixo',
```

```
'palma baixo',
'mao para baixo',
'vira a palma para baixo',
'vira para baixo',
'roda para baixo',
'pronacao',
'palm down',
'rotate down',
'pronation',
],
'forearm_neutral': [
  'antebraco neutro',
  'posicao neutra',
  'posicao normal',
  'volta ao normal',
  'voltar ao normal',
  'fica normal',
  'volta ao centro',
  'antebraco ao centro',
  'centro',
  'neutral position',
  'normal position',
  'center forearm',
  'return to center',
],
'demo': [
  'demo',
  'demonstracao',
  'faz demonstracao',
  'modo demonstracao',
  'mostra tudo',
  'testa tudo',
  'testar movimentos',
  'faz o teste',
  'modo teste',
  'demo mode',
  'show everything',
  'test movements',
  'run demo',
],
'open_index': [
  'abre o indicador',
  'abrir o indicador',
  'abre dedo indicador',
  'abre o dedo indicador',
  'abrir dedo indicador',
  'indicador aberto',
```

```
'open index',
'open index finger',
],
'close_index': [
'fecha o indicador',
'fechar o indicador',
'feche o indicador',
'fecha dedo indicador',
'fecha o dedo indicador',
'fechar dedo indicador',
'indicador fechado',
'close index',
'close index finger',
],
'open_middle': [
'abre o dedo medio',
'abrir o dedo medio',
'abre o medio',
'abre dedo medio',
'dedo medio aberto',
'open middle finger',
'open middle',
],
'close_middle': [
'fecha o dedo medio',
'fechar o dedo medio',
'feche o dedo medio',
'fecha o medio',
'fecha dedo medio',
'dedo medio fechado',
'close middle finger',
'close middle',
],
'open_ringpinky': [
'abre o anelar e o mindinho',
'abrir o anelar e o mindinho',
'abre anelar e mindinho',
'abre o mindinho e o anelar',
'anelar e mindinho abertos',
'open ring and pinky',
'open ring pinky',
],
'close_ringpinky': [
'fecha o anelar e o mindinho',
'fechar o anelar e o mindinho',
'feche o anelar e o mindinho',
'fecha anelar e mindinho',
```

```

    'fecha o mindinho e o anelar',
    'anelar e mindinho fechados',
    'close ring and pinky',
    'close ring pinky',
],
'open_thumb': [
    'abre o polegar',
    'abrir o polegar',
    'abre o dedao',
    'abre dedao',
    'polegar aberto',
    'dedao aberto',
    'open thumb',
],
'close_thumb': [
    'fecha o polegar',
    'fechar o polegar',
    'feche o polegar',
    'fecha o dedao',
    'fecha dedao',
    'polegar fechado',
    'dedao fechado',
    'close thumb',
],
};

```

/// Palavras e expressões que ajudam a pontuar cada comando possível.

```
static const Map<String, List<String>> keywords = {
```

```

    'open_hand': [
        'abrir',
        'abre',
        'aberta',
        'aberto',
        'estica',
        'esticar',
        'solta',
        'liberta',
        'estende',
        'dedos',
        'mao',
    ],
    'close_hand': [
        'fechar',
        'fecha',
        'fechada',
        'fechado',
        'punho',
    ],
};

```



```
'aperta',
'apertar',
'contrai',
'dedos',
'mao',
],
'two': [
'dois',
'2',
'dedos',
'numero',
'two',
],
'peace': [
'paz',
'vitoria',
'v',
'peace',
'victory',
],
'point': [
'apontar',
'aponta',
'indicador',
'frente',
'dedo',
'point',
'index',
'um',
],
'three': [
'tres',
'3',
'mostra',
'dedos',
'numero',
'three',
],
'four': [
'quatro',
'4',
'mostra',
'dedos',
'numero',
'four',
],
'thumbs_up': [
```

```
'joinha',
'joia',
'positivo',
'like',
'polegar',
'dedao',
'thumb',
'cima',
'fixe',
],
'ok': [
'ok',
'okay',
'certo',
'perfeito',
'bem',
'confirmar',
'confirma',
],
'wave': [
'acenar',
'acena',
'ola',
'adeus',
'cumprimenta',
'cumprimentar',
'wave',
'hello',
'goodbye',
],
'palm_up': [
'palma',
'mao',
'cima',
'roda',
'supinacao',
'up',
],
'palm_down': [
'palma',
'mao',
'baixo',
'roda',
'pronacao',
'down',
],
'forearm_neutral': [
```

```

    'neutro',
    'neutra',
    'normal',
    'centro',
    'centraliza',
    'volta',
    'voltar',
    'posicao',
    'antebraço',
    'neutral',
],
'demo': [
    'demo',
    'demonstracao',
    'mostra',
    'testa',
    'testar',
    'movimentos',
    'teste',
],
'open_index': ['abrir', 'abre', 'indicador', 'index', 'open'],
'close_index': ['fechar', 'fecha', 'indicador', 'index', 'close'],
'open_middle': ['abrir', 'abre', 'medio', 'middle', 'open'],
'close_middle': ['fechar', 'fecha', 'medio', 'middle', 'close'],
'open_ringpinky': [
    'abrir',
    'abre',
    'anelar',
    'mindinho',
    'pinky',
    'ring',
    'open',
],
'close_ringpinky': [
    'fechar',
    'fecha',
    'anelar',
    'mindinho',
    'pinky',
    'ring',
    'close',
],
'open_thumb': ['abrir', 'abre', 'polegar', 'dedao', 'thumb', 'open'],
'close_thumb': ['fechar', 'fecha', 'polegar', 'dedao', 'thumb', 'close'],
};

```

/// Palavras obrigatórias para evitar escolher comandos errados por coincidência.

```

static const Map<String, List<String>> requiredAny = {
  'open_hand': [
    'abrir',
    'abre',
    'aberta',
    'aberto',
    'estica',
    'esticar',
    'solta',
    'liberta',
    'estende',
  ],
  'close_hand': [
    'fechar',
    'fecha',
    'fechada',
    'fechado',
    'punho',
    'aperta',
    'apertar',
    'contraí',
  ],
  'two': ['dois', '2', 'two'],
  'peace': ['paz', 'vitoria', 'v', 'peace', 'victory'],
  'point': ['apontar', 'aponta', 'indicador', 'point', 'index', 'um'],
  'three': ['tres', '3', 'three'],
  'four': ['quatro', '4', 'four'],
  'thumbs_up': ['joinha', 'joia', 'positivo', 'like', 'polegar', 'dedao', 'thumb', 'fixe'],
  'ok': ['ok', 'okay', 'certo', 'perfeito', 'confirmar', 'confirma'],
  'wave': ['acenar', 'acena', 'ola', 'adeus', 'cumprimenta', 'wave', 'hello', 'goodbye'],
  'palm_up': ['cima', 'roda', 'supinacao', 'up'],
  'palm_down': ['baixo', 'roda', 'pronacao', 'down'],
  'forearm_neutral': ['neutro', 'neutra', 'normal', 'centro', 'neutral'],
  'demo': ['demo', 'demonstracao', 'mostra', 'testa', 'testar', 'teste'],
  'open_index': ['indicador', 'index'],
  'close_index': ['indicador', 'index'],
  'open_middle': ['medio', 'middle'],
  'close_middle': ['medio', 'middle'],
  'open_ringpinky': ['anelar', 'mindinho', 'pinky', 'ring'],
  'close_ringpinky': ['anelar', 'mindinho', 'pinky', 'ring'],
  'open_thumb': ['polegar', 'dedao', 'thumb'],
  'close_thumb': ['polegar', 'dedao', 'thumb'],
};

```

/// Palavras comuns que não ajudam a decidir o comando.

```

static const Set<String> fillerWords = {
  'por',

```

```
'favor',
'podes',
'pode',
'podia',
'consegue',
'consegues',
'quero',
'queria',
'faz',
'fazer',
'mete',
'coloca',
'deixa',
'bota',
'agora',
'ai',
'tipo',
'assim',
'me',
'para',
'se',
'la',
'o',
'a',
'os',
'as',
'ao',
'um',
'uma',
'e',
'pff',
'porfavor',
'please',
'can',
'you',
'make',
'do',
'the',
'an',
};
```

/// Correções simples para normalizar frases antes da comparação.

```
static const Map<String, String> replacements = {
  'maun': 'mao',
  'mão': 'mao',
  'maos': 'mao',
  'mãos': 'mao',
}
```

```

'fexar': 'fechar',
'fexa': 'fecha',
'feira': 'fecha',
'feixar': 'fechar',
'jóinha': 'joinha',
'joínha': 'joinha',
'join': 'joinha',
'jóia': 'joia',
'dedão': 'dedao',
'médio': 'medio',
'meio': 'medio',
'indicado': 'indicador',
'indica dor': 'indicador',
'vitória': 'vitoria',
'três': 'tres',
'tree': 'three',
'palm': 'palma',
'pra': 'para',
'xau': 'adeus',
'chau': 'adeus',
'tá': 'esta',
'ta': 'esta',
'num': 'numero',
};

```

/// Recebe uma frase e devolve a ação mais provável.

```

static String classify(String text) {
    final cleanText = normalize(text);

    if (cleanText.isEmpty) {
        return 'unknown';
    }

    final scores = <String, double>{};

    for (final command in strongPhrases.keys) {
        scores[command] = _scoreCommand(cleanText, command);
    }

    final ranked = scores.entries.toList()
        ..sort((a, b) => b.value.compareTo(a.value));

    final best = ranked[0];
    final second =
        ranked.length > 1 ? ranked[1] : const MapEntry('unknown', 0.0);

    if (best.value < 2.2) {

```

```

    return 'unknown';
}

if (second.value > 0 && best.value - second.value < 0.45) {
    return 'unknown';
}

return best.key;
}

/// Limpa acentos, pontuação e espaços para facilitar a comparação.
static String normalize(String text) {
    var value = text.toLowerCase().trim();

    value = _removeAccents(value);

    for (final entry in replacements.entries) {
        final wrong = _removeAccents(entry.key.toLowerCase());
        final right = _removeAccents(entry.value.toLowerCase());
        value = value.replaceAll(RegExp("\\b${RegExp.escape(wrong)}\\b"), right);
    }

    value = value.replaceAll(RegExp(r'^a-z0-9\\s'), ' ');
    value = value.replaceAll(RegExp(r'\\s+'), ' ').trim();

    return value;
}

/// Divide a frase em palavras úteis para o cálculo de pontuação.
static List<String> _tokens(String text) {
    return normalize(text)
        .split(' ')
        .where((word) => word.isNotEmpty && !fillerWords.contains(word))
        .toList();
}

/// Calcula a pontuação de um comando com base em frases, palavras e semelhança.
static double _scoreCommand(String cleanText, String command) {
    final inputTokens = _tokens(cleanText);
    var score = 0.0;

    final phrases = strongPhrases[command] ?? [];
    final commandKeywords = keywords[command] ?? [];

    for (final phrase in phrases) {
        final cleanPhrase = normalize(phrase);

```

```

if (cleanText.contains(cleanPhrase)) {
    score += 3.0;
}

final phraseScore = _similarity(cleanText, cleanPhrase);

if (phraseScore >= 0.75) {
    score += phraseScore * 2.5;
}

final overlap = _tokenOverlapScore(inputTokens, _tokens(cleanPhrase));
score += overlap * 2.0;
}

for (final keyword in commandKeywords) {
    final cleanKeyword = normalize(keyword);

    for (final word in inputTokens) {
        final sim = _similarity(word, cleanKeyword);

        if (word == cleanKeyword) {
            score += 0.9;
        } else if (sim >= 0.82) {
            score += 0.6;
        } else if (sim >= 0.72) {
            score += 0.3;
        }
    }
}

if (!_containsRequiredWord(inputTokens, requiredAny[command] ?? [])) {
    score *= 0.35;
}

final joined = inputTokens.join(' ');
final fingerWords = [
    'indicador',
    'medio',
    'anelar',
    'mindinho',
    'polegar',
    'dedao',
    'index',
    'middle',
    'ring',
    'pinky',
    'thumb',

```



```

];

if (command == 'open_hand' &&
    _hasAny(joined, ['fechar', 'fecha', 'punho', 'fechada'])) {
    score *= 0.25;
}

if (command == 'close_hand' &&
    _hasAny(joined, ['abrir', 'abre', 'aberta', 'estica'])) {
    score *= 0.25;
}

if ((command == 'open_hand' || command == 'close_hand') &&
    _hasAny(joined, fingerWords) &&
    !_hasAny(joined, ['todos', 'tudo', 'mao', 'dedos'])) {
    score *= 0.25;
}

if (command == 'palm_up' && joined.contains('baixo')) {
    score *= 0.1;
}

if (command == 'palm_down' && joined.contains('cima')) {
    score *= 0.1;
}

if (command == 'peace' && _hasAny(joined, ['dois', '2', 'tres', '3', 'quatro', '4'])) {
    score *= 0.3;
}

if (command == 'two' && _hasAny(joined, ['paz', 'vitoria', 'tres', '3', 'quatro', '4'])) {
    score *= 0.3;
}

if (command == 'three' &&
    _hasAny(joined, ['dois', '2', 'paz', 'vitoria', 'quatro', '4'])) {
    score *= 0.3;
}

if (command == 'four' && _hasAny(joined, ['dois', '2', 'tres', '3', 'paz', 'vitoria'])) {
    score *= 0.3;
}

if (command == 'thumbs_up' && joined.contains('palma')) {
    score *= 0.35;
}

```

```

if (command == 'palm_up' && _hasAny(joined, ['joinha', 'polegar', 'thumb'])) {
    score *= 0.35;
}

if (command.startsWith('open_') &&
    !command.contains('hand') &&
    _hasAny(joined, ['fechar', 'fecha', 'feche'])) {
    score *= 0.3;
}

if (command.startsWith('close_') &&
    !command.contains('hand') &&
    _hasAny(joined, ['abrir', 'abre'])) {
    score *= 0.3;
}

return score;
}

/// Confirma se a frase tem pelo menos uma palavra obrigatória para o comando.
static bool _containsRequiredWord(
    List<String> inputTokens,
    List<String> requiredWords,
) {
    if (requiredWords.isEmpty) return true;

    for (final required in requiredWords) {
        final cleanRequired = normalize(required);

        for (final word in inputTokens) {
            if (word == cleanRequired || _similarity(word, cleanRequired) >= 0.82) {
                return true;
            }
        }
    }

    return false;
}

/// Verifica se alguma expressão de uma lista aparece no texto.
static bool _hasAny(String text, List<String> words) {
    return words.any(text.contains);
}

/// Mede quantas palavras importantes coincidem entre duas expressões.
static double _tokenOverlapScore(
    List<String> inputTokens,

```

```

List<String> patternTokens,
) {
    if (patternTokens.isEmpty) return 0.0;

    var score = 0.0;

    for (final pattern in patternTokens) {
        var best = 0.0;

        for (final token in inputTokens) {
            best = max(best, _similarity(token, pattern));
        }

        if (best >= 0.82) {
            score += 1.0;
        } else if (best >= 0.68) {
            score += 0.6;
        }
    }

    return score / patternTokens.length;
}

/// Calcula uma semelhança simples entre duas frases.
static double _similarity(String a, String b) {
    if (a == b) return 1.0;
    if (a.isEmpty || b.isEmpty) return 0.0;

    final matrix = List.generate(
        a.length + 1,
        (_) => List<int>.filled(b.length + 1, 0),
    );

    for (var i = 0; i <= a.length; i++) {
        matrix[i][0] = i;
    }

    for (var j = 0; j <= b.length; j++) {
        matrix[0][j] = j;
    }

    for (var i = 1; i <= a.length; i++) {
        for (var j = 1; j <= b.length; j++) {
            final cost = a[i - 1] == b[j - 1] ? 0 : 1;

            matrix[i][j] = min(
                min(

```

```

        matrix[i - 1][j] + 1,
        matrix[i][j - 1] + 1,
    ),
    matrix[i - 1][j - 1] + cost,
);
}
}

final distance = matrix[a.length][b.length];
final maxLength = max(a.length, b.length);

return 1.0 - distance / maxLength;
}

/// Remove acentos para comparar português e inglês de forma mais estável.
static String _removeAccents(String text) {
    const from = 'áàãâäëèêëîïíîóôõöùúûüçñ';
    const to = 'aaaaaeeeeeiioooooouuuuch';

    var result = text;

    for (var i = 0; i < from.length; i++) {
        result = result.replaceAll(from[i], to[i]);
    }

    return result;
}
}

```

Diretório – proj_armiac/lib/settings/settings_page.dart

```

import 'package:flutter/material.dart';
import 'package:package_info_plus/package_info_plus.dart';

import '../app_language.dart';
import '../l10n.dart';
import 'terms_page.dart';

class SettingsPage extends StatefulWidget {
    const SettingsPage({super.key});

    @override
    State<SettingsPage> createState() => _SettingsPageState();
}

```

/// Carrega dados da aplicação e apresenta opções de idioma e ajuda.

```

class _SettingsPageState extends State<SettingsPage> {
  String appVersion = "";

  @override
  /// Carrega a versão assim que a página é aberta.
  void initState() {
    super.initState();
    _loadVersion();
  }

  /// Obtém a versão definida no pacote da aplicação.
  Future<void> _loadVersion() async {
    final info = await PackageInfo.fromPlatform();
    if (!mounted) return;
    setState(() => appVersion = info.version);
  }

  /// Cria cartões informativos reutilizáveis.
  Widget _infoTile({
    required IconData icon,
    required String title,
    required String subtitle,
  }) {
    return Card(
      child: ListTile(
        leading: Icon(icon),
        title: Text(title, style: const TextStyle(fontWeight: FontWeight.bold)),
        subtitle: Text(subtitle),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    return AnimatedBuilder(
      animation: appSettings,
      builder: (context, _) {
        final l10n = L10n(appSettings.language);

        return ListView(
          padding: const EdgeInsets.all(16),
          children: [
            Text(l10n.settings, style: const TextStyle(fontSize: 24, fontWeight:
FontWeight.bold)),
            const SizedBox(height: 16),
            Card(
              child: ListTile(

```

```

        leading: ClipRRect(
          borderRadius: BorderRadius.circular(10),
          child: Image.asset('assets/images/Armiac_LOGO.png', width: 48, height: 48, fit:
BoxFit.contain),
        ),
        title: Text(l10n.appName, style: const TextStyle(fontWeight: FontWeight.bold)),
        subtitle: Text(l10n.appSubtitle),
      ),
    ),
    Card(
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Text(l10n.languageLabel, style: const TextStyle(fontWeight: FontWeight.bold)),
            const SizedBox(height: 12),
            SegmentedButton<AppLanguage>(
              segments: [
                ButtonSegment(
                  value: AppLanguage.portuguese,
                  label: Text(l10n.portuguese),
                  icon: const Text('PT'),
                ),
                ButtonSegment(
                  value: AppLanguage.english,
                  label: Text(l10n.english),
                  icon: const Text('GB'),
                ),
              ],
              selected: {appSettings.language},
              onSelectionChanged: (selection) {
                appSettings.setLanguage(selection.first);
              },
            ),
            const SizedBox(height: 10),
            Text(l10n.languageHelp, style: const TextStyle(color: Colors.grey)),
          ],
        ),
      ),
    ),
    _infoTile(
      icon: Icons.mic,
      title: l10n.microphonePermission,
      subtitle: l10n.microphonePermissionBody,
    ),
    _infoTile(

```

```

        icon: Icons.wifi,
        title: l10n.armiacConnection,
        subtitle: l10n.armiacConnectionBody,
      ),
      const SizedBox(height: 12),
      Text(
        '${l10n.appVersion}: $appVersion',
        style: const TextStyle(fontSize: 16, color: Colors.grey),
      ),
      const SizedBox(height: 24),
      Center(
        child: TextButton.icon(
          onPressed: () => Navigator.push(
            context,
            MaterialPageRoute(builder: (_) => const TermsPage()),
          ),
          icon: const Icon(Icons.open_in_new),
          label: Text(l10n.serviceTerms),
        ),
      ),
    ],
  );
}

```

Diretório – proj_armiac/lib/settings/terms_page.dart

```

import 'package:flutter/material.dart';

import '../app_language.dart';
import '../l10n.dart';

/// Página com os termos de serviço.
class TermsPage extends StatefulWidget {
  const TermsPage({super.key});

  @override
  State<TermsPage> createState() => _TermsPageState();
}

/// Controla as animações de entrada do texto dos termos.
class _TermsPageState extends State<TermsPage> with SingleTickerProviderStateMixin {
  late final AnimationController _controller;
  late final Animation<double> _fade;
  late final Animation<Offset> _slide;

```

```

@override
/// Prepara e inicia a animação de entrada.
void initState() {
  super.initState();
  _controller = AnimationController(vsync: this, duration: const Duration(milliseconds:
450));
  _fade = CurvedAnimation(parent: _controller, curve: Curves.easeOut);
  _slide = Tween<Offset>(begin: const Offset(0, 0.04), end: Offset.zero).animate(_fade);
  _controller.forward();
}

```

```

@override
/// Liberta o controlador da animação.
void dispose() {
  _controller.dispose();
  super.dispose();
}

```

```

@override
Widget build(BuildContext context) {
  return AnimatedBuilder(
    animation: appSettings,
    builder: (context, _) {
      final l10n = L10n(appSettings.language);
      final cs = Theme.of(context).colorScheme;

      return Scaffold(
        appBar: AppBar(
          title: Text(l10n.termsTitle),
          backgroundColor: cs.inversePrimary,
        ),
        body: FadeTransition(
          opacity: _fade,
          child: SlideTransition(
position: _slide,
            child: ListView(
              padding: const EdgeInsets.fromLTRB(16, 16, 16, 24),
              children: [
                Container(
                  padding: const EdgeInsets.all(16),
                  decoration: BoxDecoration(
                    borderRadius: BorderRadius.circular(18),
                    gradient: LinearGradient(
                      colors: [
                        cs.primary.withOpacity(0.12),
                        cs.secondary.withOpacity(0.10),
                      ],

```



```

        begin: Alignment.topLeft,
        end: Alignment.bottomRight,
    ),
    border: Border.all(color: cs.primary.withOpacity(0.18)),
),
child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
        Text(
            l10n.termsHeader,
            style: const TextStyle(fontSize: 18, fontWeight: FontWeight.w900),
        ),
        const SizedBox(height: 8),
        Row(
            children: [
                Icon(Icons.event, size: 18, color: cs.primary),
                const SizedBox(width: 8),
                Text(
                    l10n.lastUpdate,
                    style: const TextStyle(fontSize: 13.5, fontWeight: FontWeight.w600),
                ),
            ],
        ),
    ],
),
const SizedBox(height: 18),
...l10n.termsParagraphs.map(
    (text) => Padding(
        padding: const EdgeInsets.only(bottom: 12),
        child: Text(text, style: const TextStyle(fontSize: 15.5, height: 1.45)),
    ),
),
],
),
),
),
);
},
);
}
}

```

Código pubspec.yaml:

Diretório - proj_armiac/pubspec.yaml

name: proj_armiac
description: "Desenvolvido por Quévin Tavares"

publish_to: 'none'

version: 1.1.7+9

environment:
 sdk: ">=3.10.0 <4.0.0"

dependencies:
 flutter:
 sdk: flutter
 cupertino_icons: ^1.0.8
 package_info_plus: ^8.0.2
 http: ^1.2.2
 speech_to_text: ^7.3.0
 flutter_tts: ^4.2.0
 shared_preferences: ^2.3.2

dev_dependencies:

flutter_test:
 sdk: flutter
 flutter_lints: ^6.0.0

flutter:
 uses-material-design: true
 assets:
 - assets/images/Armiac_LOGO.png
- assets/tutorial

7.2 – APK da aplicação ARMIAC

APK da APP:

https://drive.google.com/file/d/1pG642exOyXLrIG7X_opwh_tH_ayCOEfX/view?usp=sharing

7.3 – Links dos vídeos do Youtube

- **Exemplo de projetos base:**

1º projeto:

<https://youtu.be/L2ddw3MQI9g?si=44-PRt1J8Qk6TCc3>

2º projeto:

https://www.tiktok.com/@pathofseb/video/7523183373288869133?is_from_webapp=1&sender_device=pc

- **Impressão 3D:**

<https://youtube.com/shorts/XccsXvw3TjE?si=Ub7Kg75nmciBa69N>

- **Primeiro teste com servomotor:**

<https://youtube.com/shorts/2wemvdqr7-U?si=sgkJaba-hj45bRa1>

- **Demonstração do funcionamento dos dedos:**

https://youtube.com/shorts/j9Mxo_9-ovw?si=rJqrfvmzXTgFEHxD

- **Demonstração do braço:**

<https://youtube.com/shorts/-pQoMxjrOXw?si=P2gNtikCYM4VFR2T>

- **Demonstração do projeto inacabado:**

1º vídeo:

https://youtu.be/dLHhLJuH_1o?si=sjqv0j5o4DHVVhbd

2º vídeo:

<https://youtube.com/shorts/C5wIR47ePRw?si=5v5-RfpaooyCxAZo>

- **Demonstração do projeto com bateria vs com fonte:**

<https://youtu.be/RV1m9Q8S5C0?si=15LIKdADrWPj9K7f>